Butterfly-accelerated manifold harmonic transforms

Samuel F. Potter

Courant Institute of Mathematical Sciences
New York University

About me

- PhD in computer science at UMD in 2021
- Main research interest: “fast algorithms”
- I like working on “real” industrial problems
- Dissertation on fast algorithms for solving the eikonal equation with applications in computational room acoustics and spatial audio
- Currently Courant Instructor at NYU’s Courant Institute
- Working on butterfly algorithms, primarily for high-frequency acoustic scattering

“Fast algorithms”

- Goal: solve a PDE or an integral equation in “optimal time” (i.e., time and space complexity are ideally close to $O(N)$ if N is the number of degrees of freedom)
- Classic examples: the fast Fourier transform, multigrid, etc.
- My expertise: marching methods, hierarchical matrix methods, fast multipole method, and now butterfly algorithms
- Contrary to “theoretical computer science”: a fast algorithm should have good constants so that we can use it to solve problems!

Joint work with. . .



Tim Langlois
Adobe Inc.



Mike O’Neil
CIMS



Hamish Potter
IIT

What's this talk about?

A new algorithm:

A fast algorithm based on the butterfly factorization of the "manifold harmonic transform" (linear transform into spectrum of discretized Laplace-Beltrami operator)...

Why do we care?

There's no fast Fourier transform for an unstructured domain. Lots of applications could take advantage of one in one way or another.

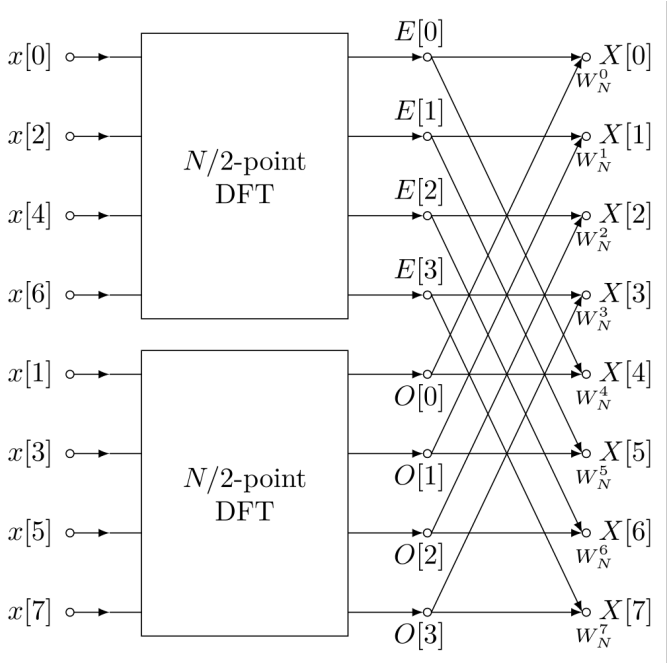
History of the butterfly factorization

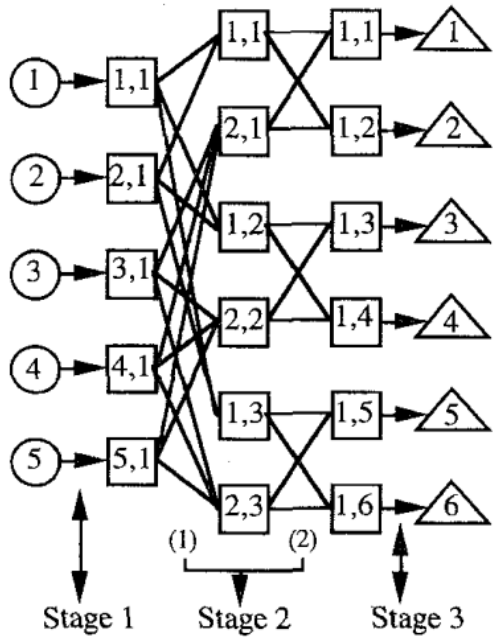
- **1996:** Michielssen and Boag introduce the butterfly factorization for problems in electromagnetic scattering, which turns out to be of broader applicability.
- **2007–10:** O'Neil, Rokhlin, et al. use the butterfly factorization to systematically and rigorously develop fast special function transforms.
- **2010–now:** Extension to higher (3!) dimensions, application to Fourier integral operators, fast direct solvers, etc.

Authors involved: Candes, Demanet, Engquist, Michielssen, Poulson, Ho, Yang, Ying, others.

What is the butterfly factorization?

- Linear-algebraic generalization of the FFT
- Can be used to compress other special function transforms and oscillatory kernel matrices





Left: Cooley-Tukey FFT algorithm - Wikipedia.

Right: Michielssen et al. “A multilevel matrix decomposition algorithm for analyzing scattering from large structures.”

When can we butterfly a matrix?

. . . when it satisfies the **butterfly property**:

An $n \times n$ matrix A can be butterfly factorized into a product of $O(\log n)$ factors with $O(n)$ nonzero entries each if the rows and columns can be permuted such that the rank of contiguous subblocks is proportional to the product of the "area" of the rows and the "area" of the columns (the "space-frequency product").

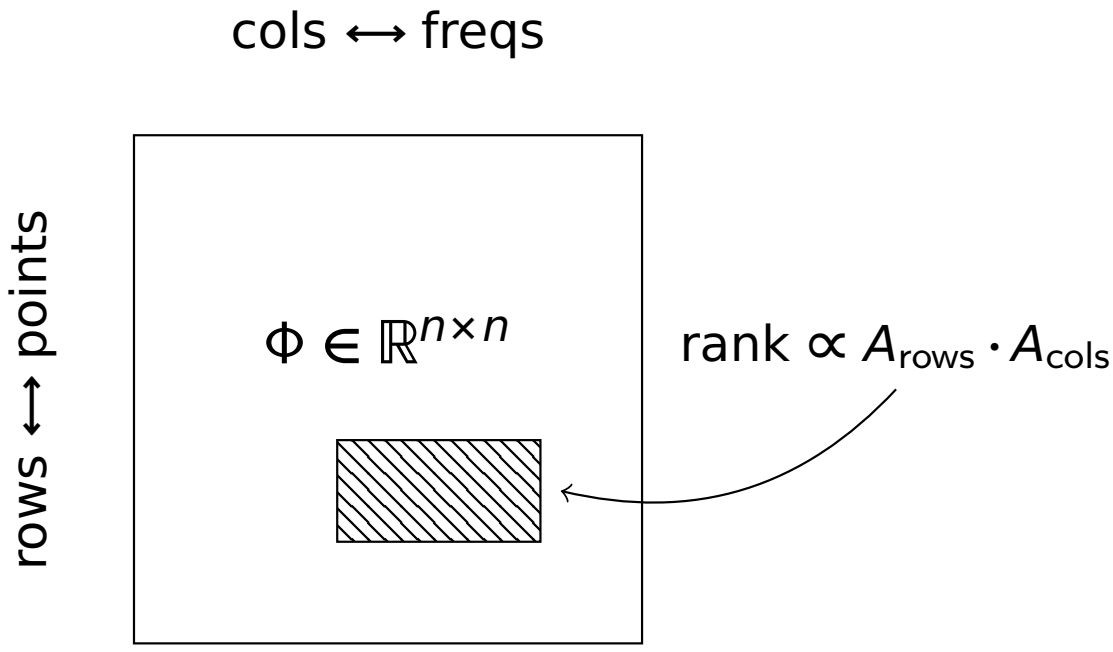
When can we butterfly a matrix?

. . . when it satisfies the **butterfly property**:

> An $n \times n$ matrix A can be butterfly factorized into a product of $O(\log n)$ factors with $O(n)$ nonzero entries each if the rows and columns can be permuted such that the rank of contiguous subblocks is proportional to the product of the "area" of the rows and the "area" of the columns (the "space-frequency product").

???

When can we butterfly a matrix?

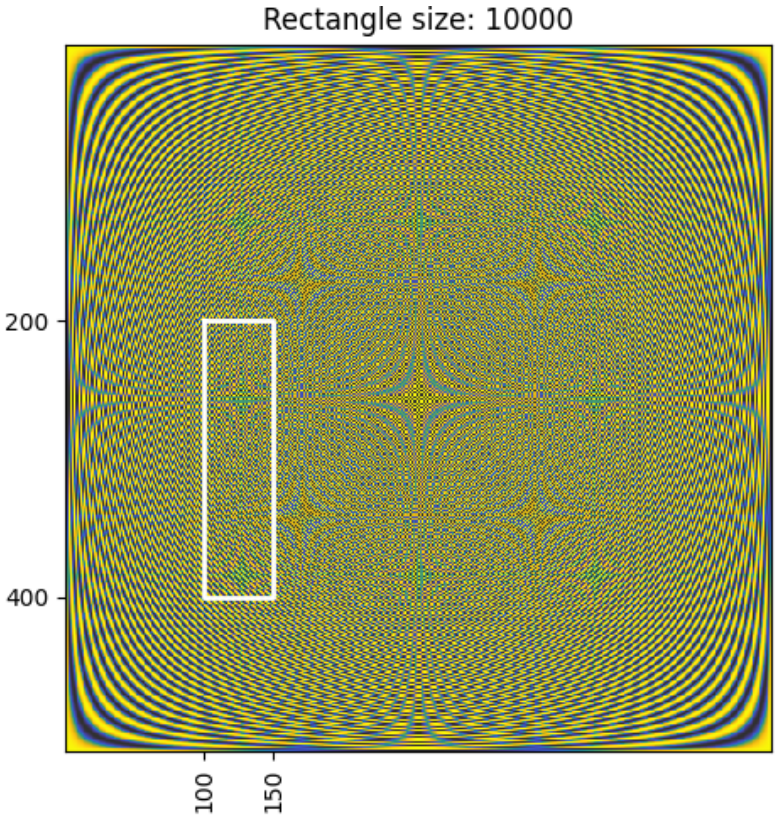


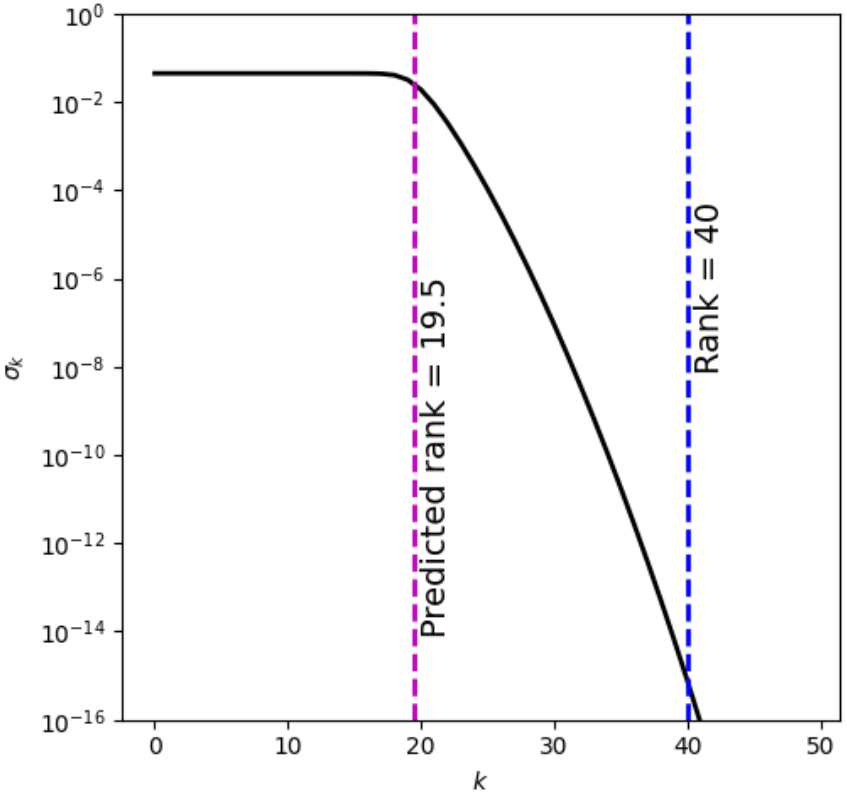
Goal: construct space and frequency trees so that as we move up and down them simultaneously we keep the space-frequency product constant

Example: the discrete Fourier transform

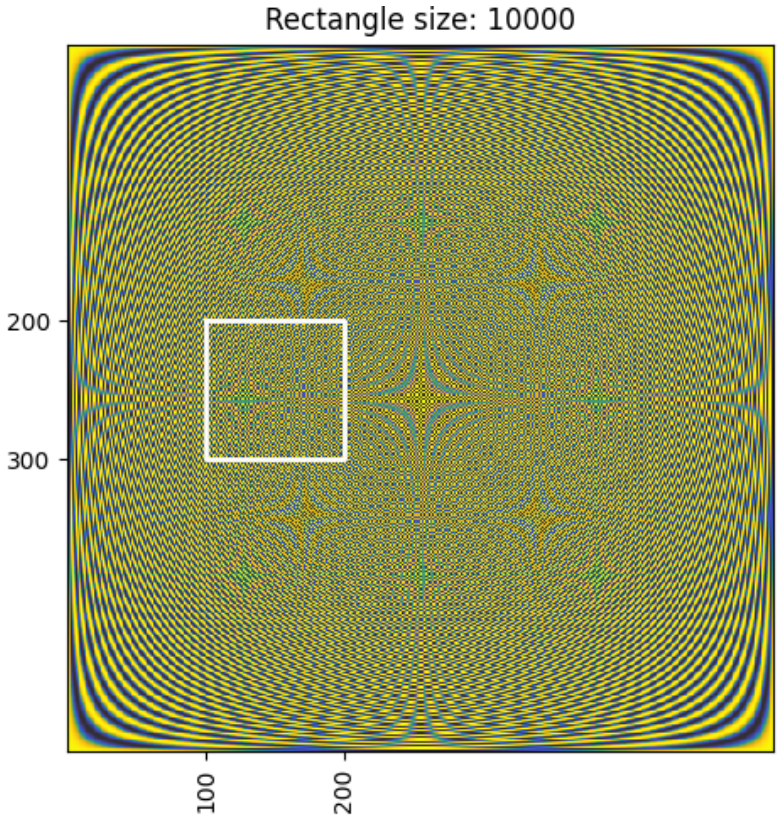
- The DFT: $X \in \mathbb{C}^{N \times N}$, where $X_{kn} = e^{i2\pi kn/N}$
- Easy to prove that the DFT satisfies the butterfly property
- The fast Fourier transform: use algebraic properties of the DFT to apply it in $O(N \log N)$ operations
- Can view the FFT as a sparse matrix factorization of the DFT into $O(\log N)$ sparse factors—so, interpret it as a butterfly factorization
- Can also compute "the" butterfly factorization of the DFT, but very inefficient! (Constant is ~300x worse)

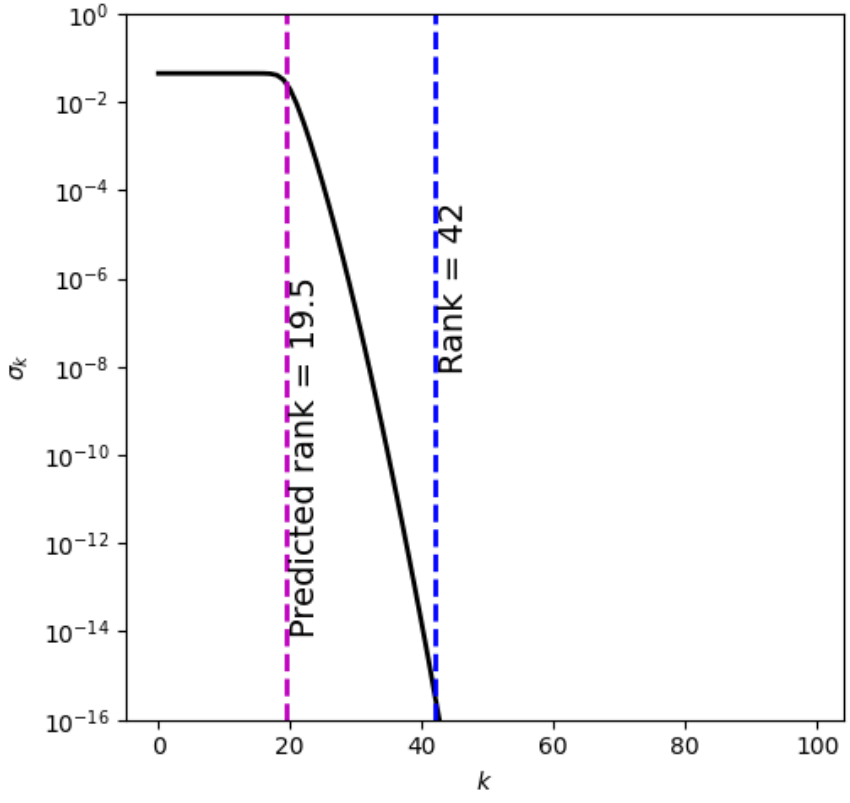
The DFT can be “butterflied”



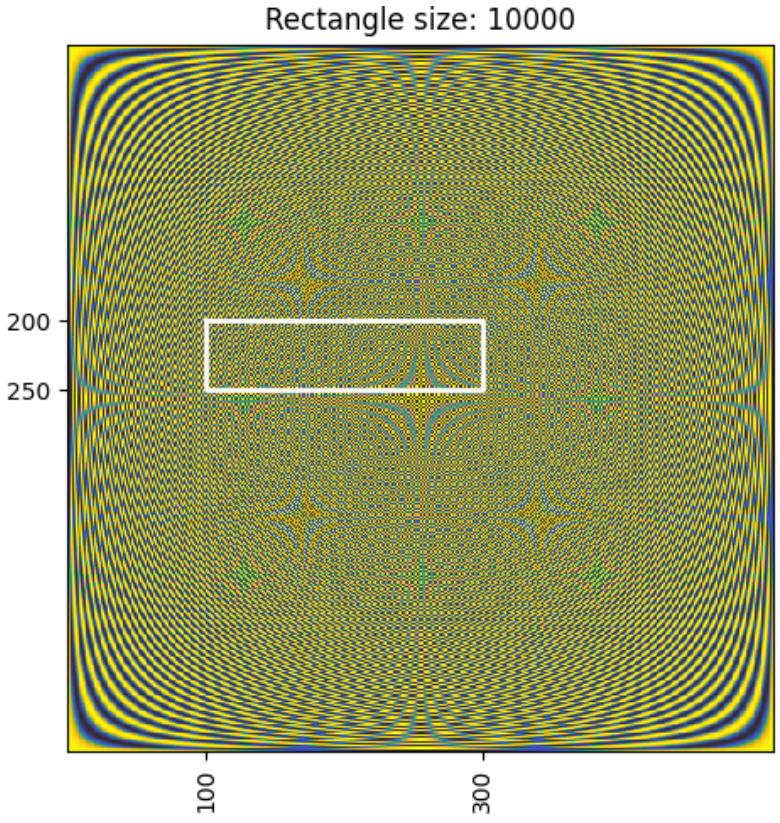


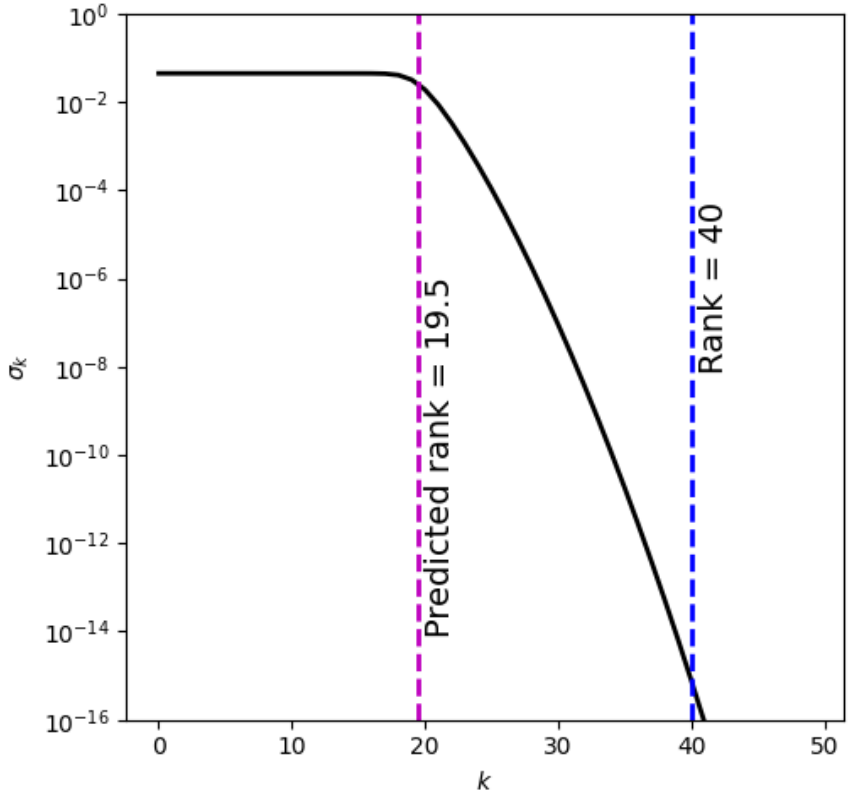
The DFT can be “butterflied”



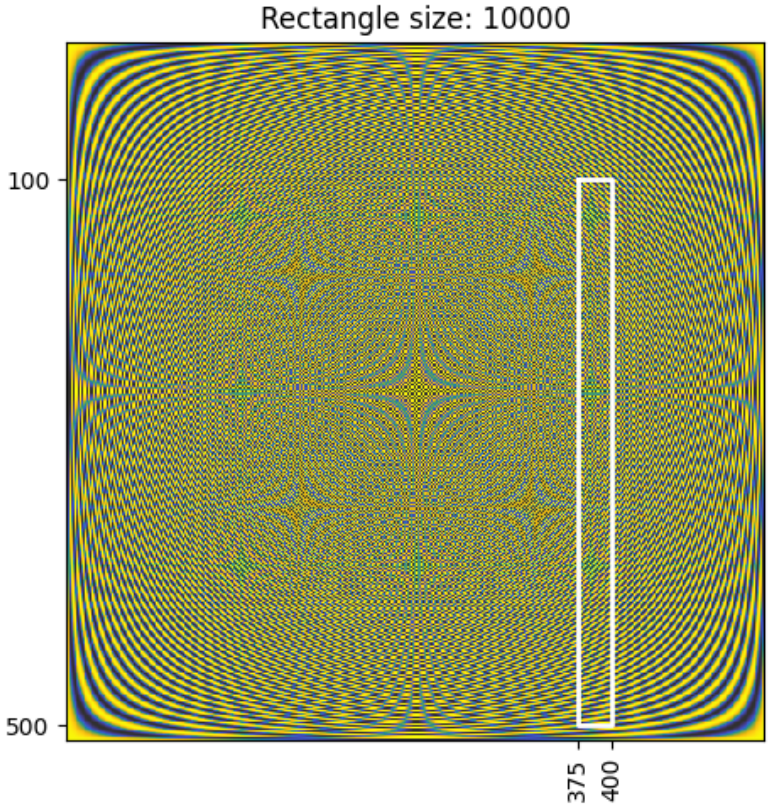


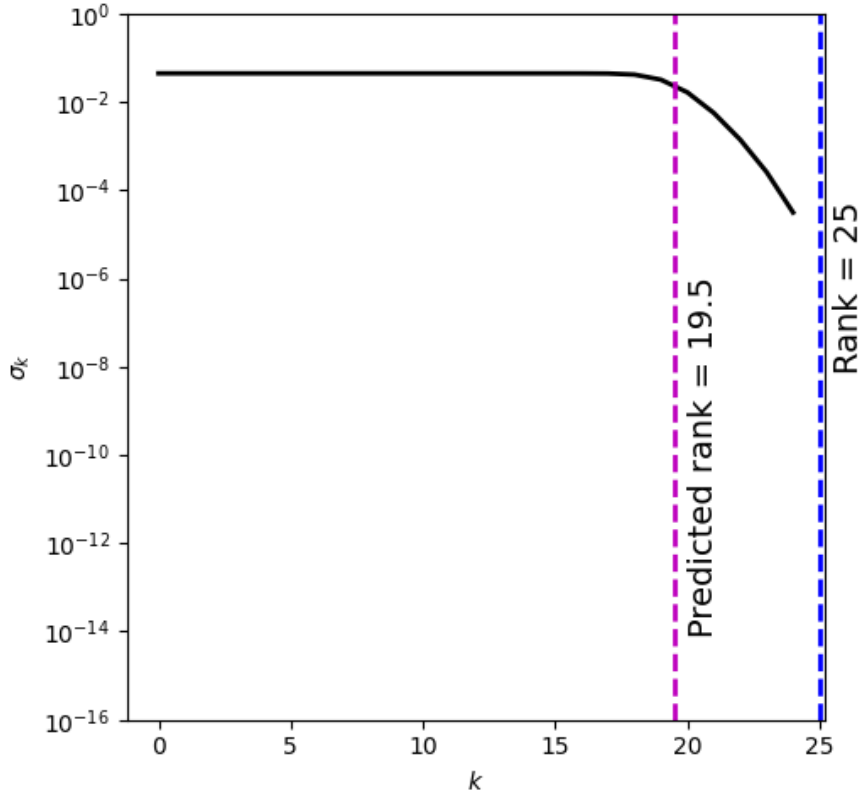
The DFT can be “butterflied”



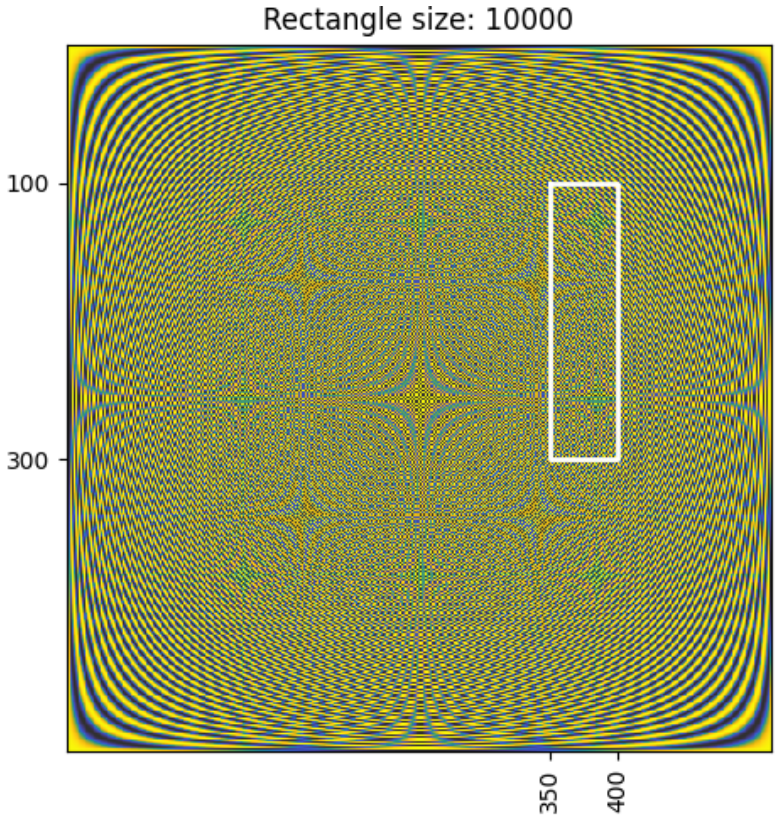


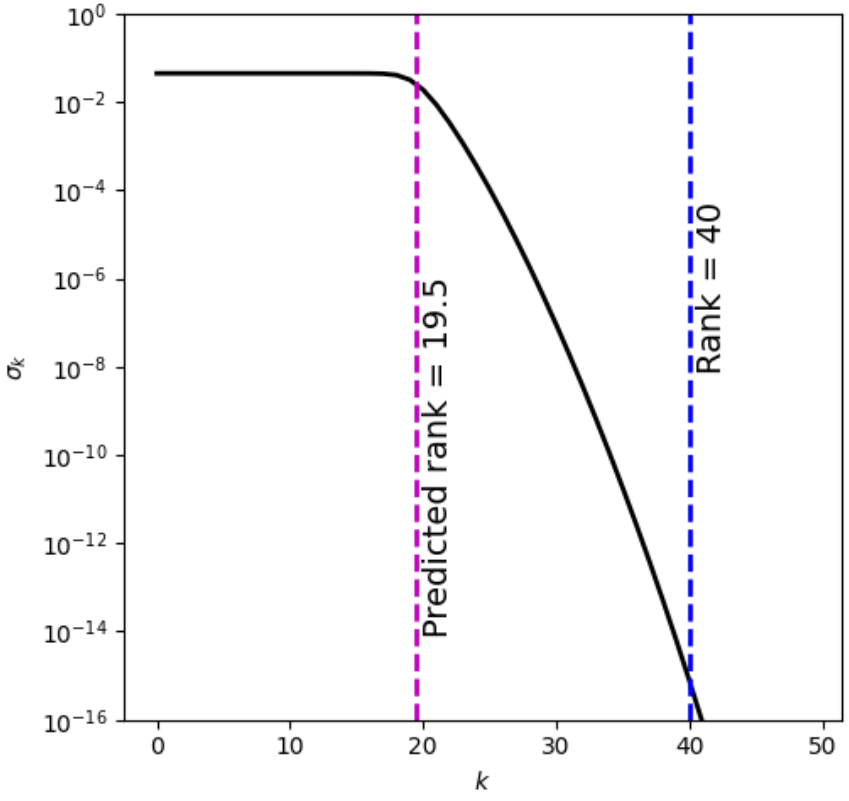
The DFT can be “butterflied”



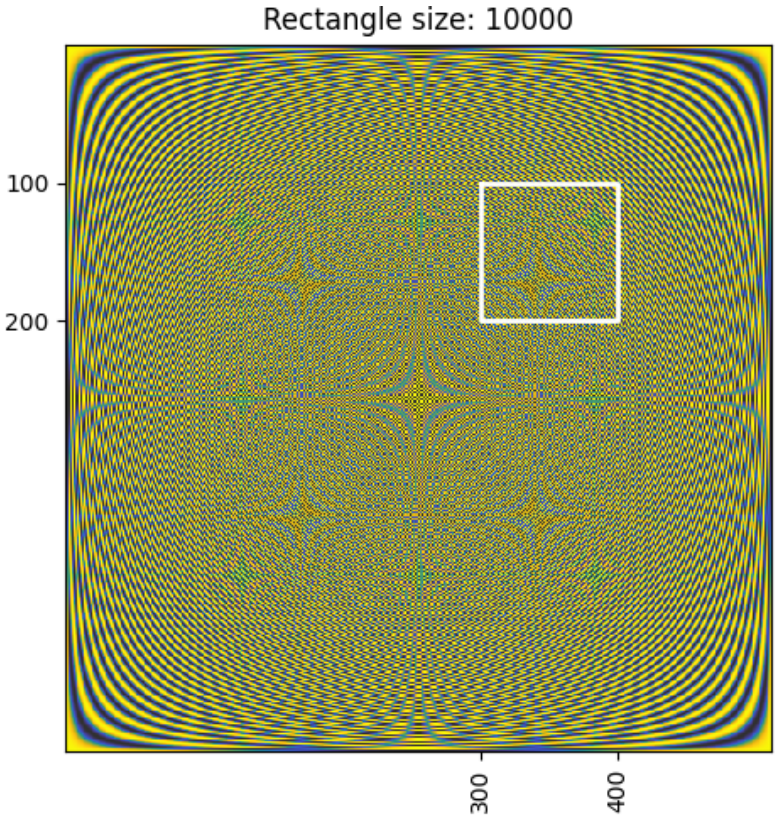


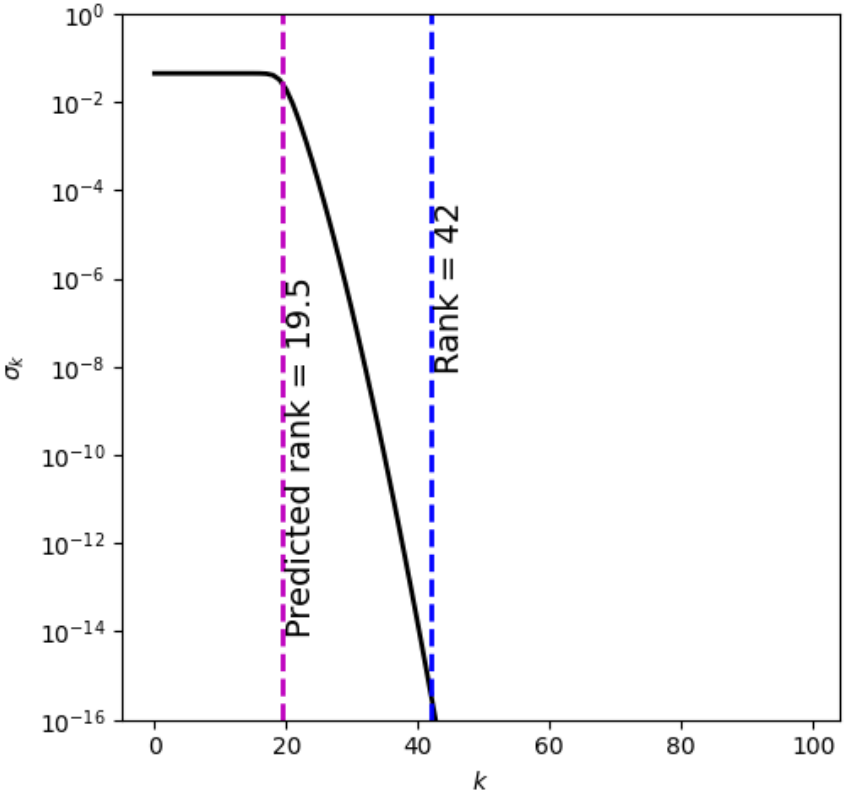
The DFT can be “butterflied”



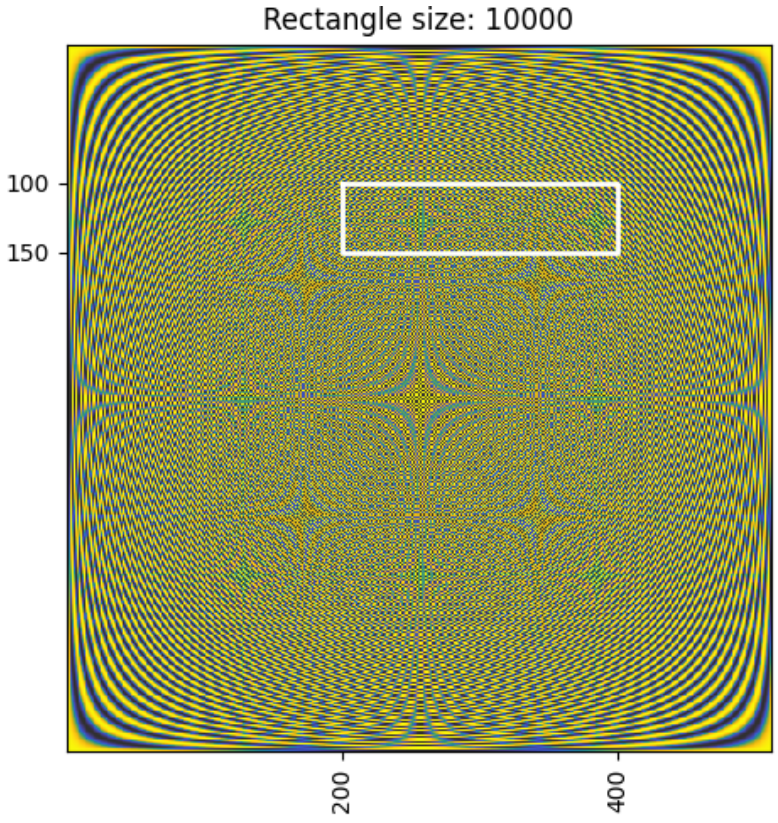


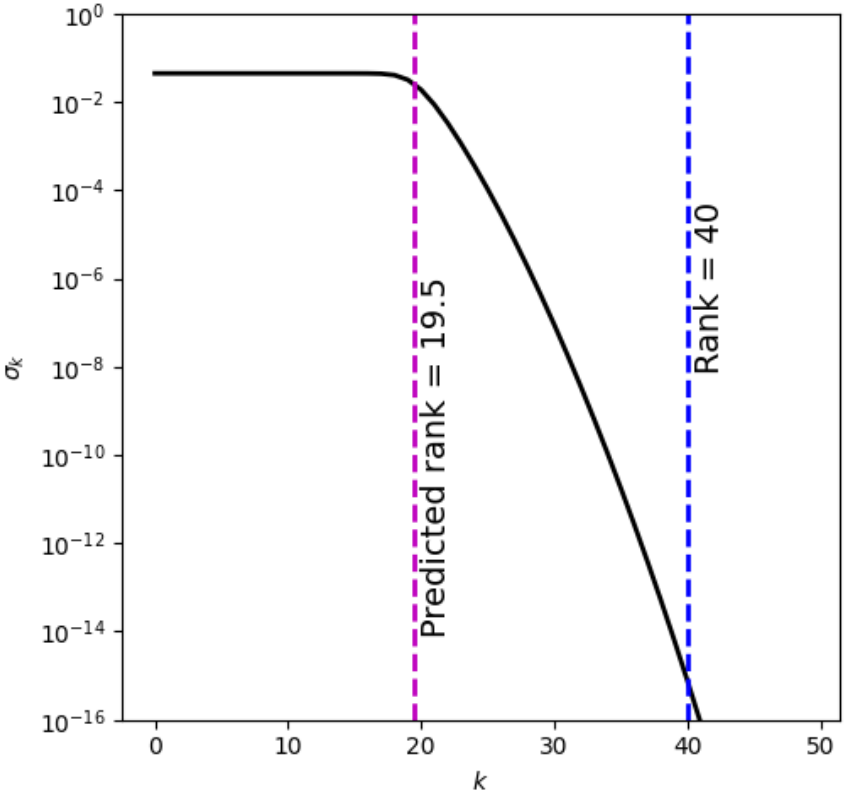
The DFT can be “butterflied”



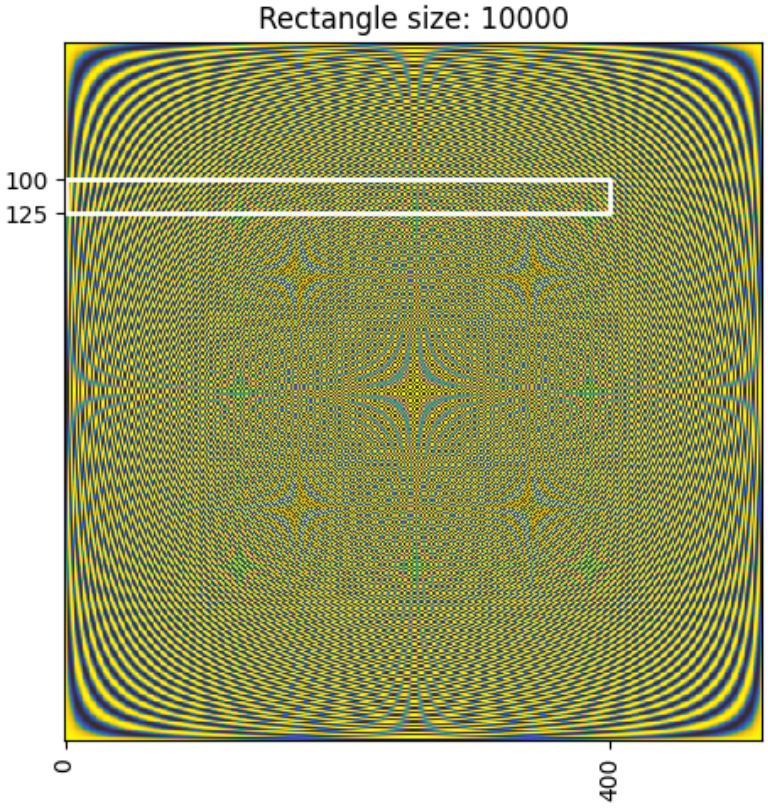


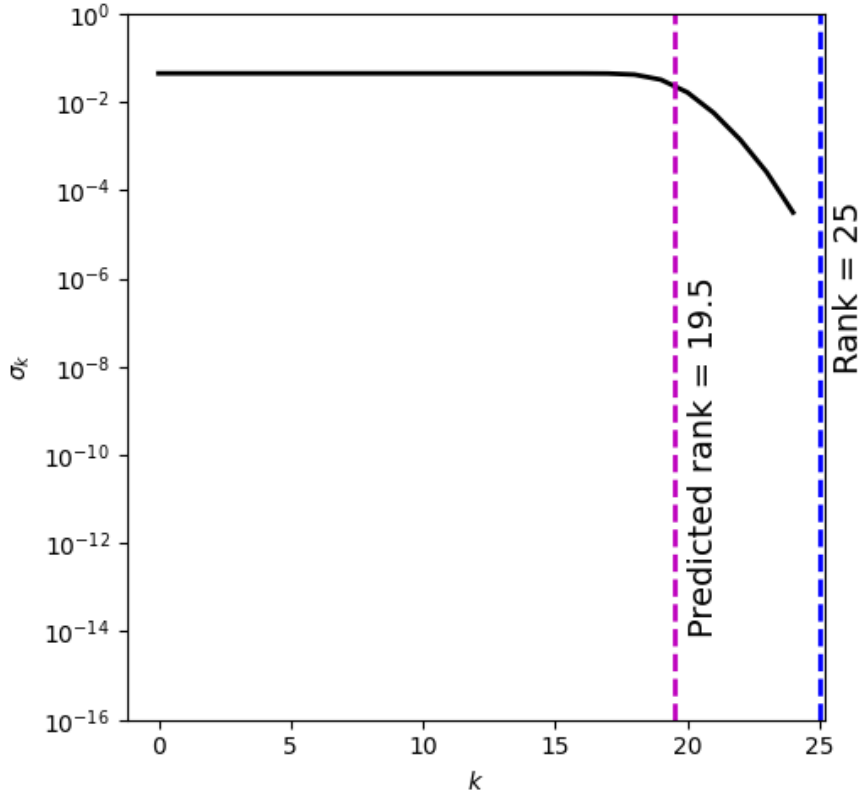
The DFT can be “butterflied”





The DFT can be “butterflied”





What are “manifold harmonics”?

- Let M be a compact Riemannian manifold. The **Laplace-Beltrami operator** is the Laplacian on M:

$$\Delta_M = \nabla_M \cdot \nabla_M$$

- A **manifold harmonic** is an eigenfunction of the Laplace-Beltrami eigenproblem on M:

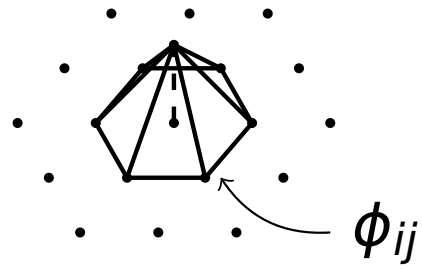
$$\Delta_M u = \lambda u, \qquad u \in L^2(M)$$

Focus of this talk: genus 0 surfaces $S \subseteq \mathbb{R}^3$.

Why do we care?

- Eigfuncs of Δ_M give orthonormal basis for $L^2(M)$
- Useful for function compression
- Useful for signal processing
- Useful for geometry processing
- Useful for solving PDEs

How do we compute manifold harmonics?



- Discretize $S \rightarrow$ tri. mesh with vertices $V = \{x_i\}_{i \in I}$
- Set up basis of linear finite elements $\{\phi_i\}_{i \in I}$:

$$\phi_i(x_j) = \delta_{ij}, \qquad x_j \in V, \qquad \sum_i u_i \phi_i(x) \in C^0(S)$$

- Leads to generalized eigenvalue problem:

$$Lu = \lambda M u$$

  where L and M are stiffness and mass matrices.
- Solve using standard sparse eigenvalue package.

How do we compute manifold harmonics?

"Asian dragon" mesh: 56.4K vertices



Source: Stanford 3D Scanning Repository

How do we compute manifold harmonics?

A harmonic ($\lambda \approx 0.848$) on the “Asian dragon” mesh



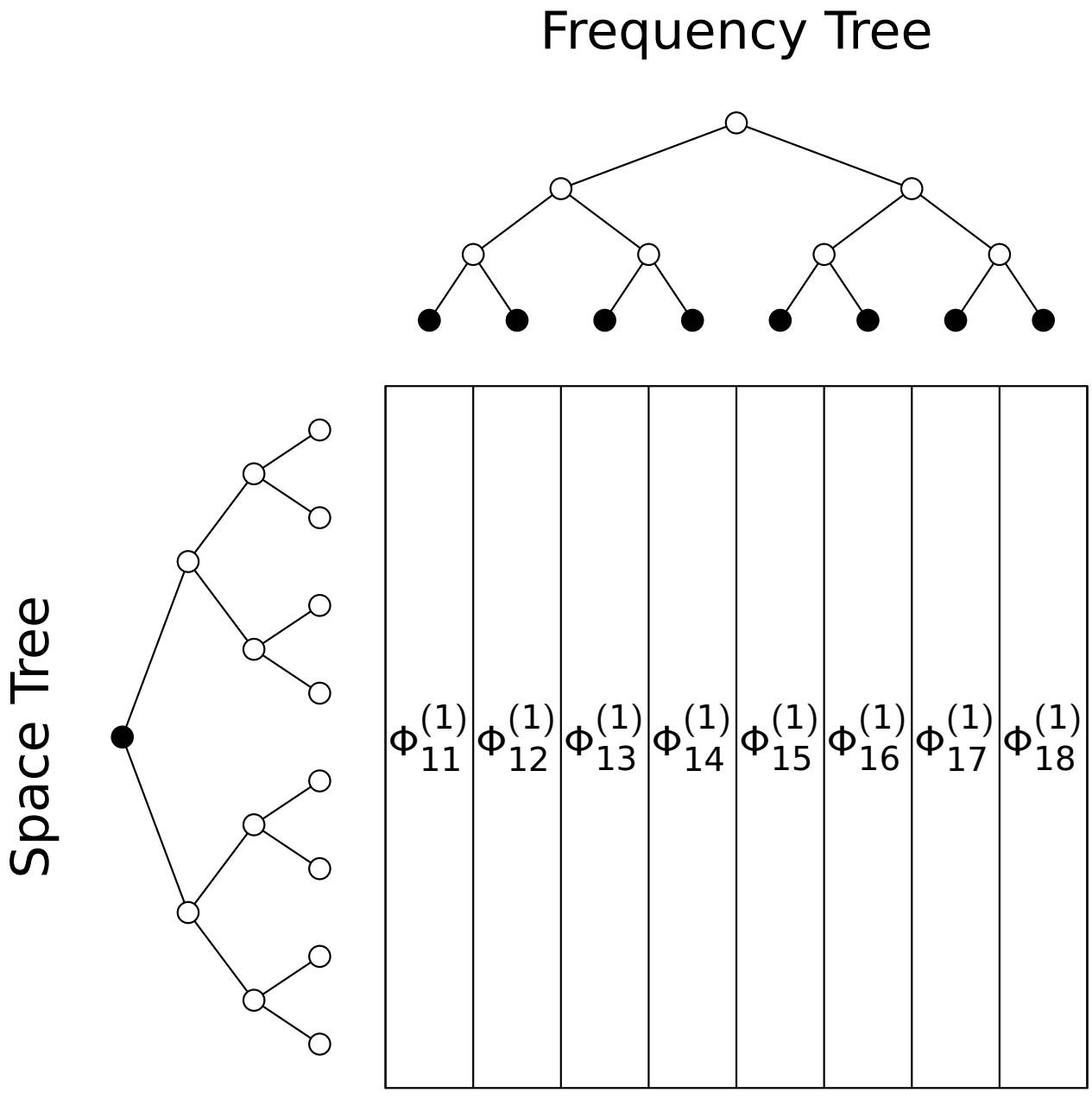
Source: Stanford 3D Scanning Repository

Spatial partitioning

"Asian dragon" with **octree ordering** on vertices

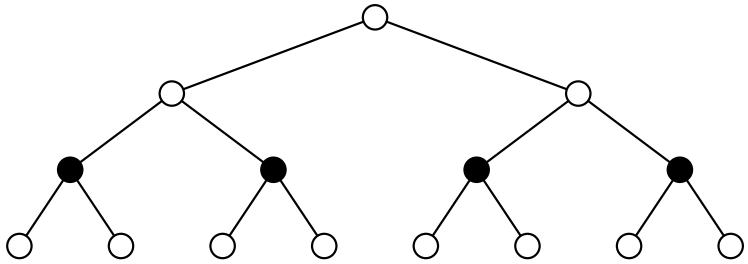


The butterfly property

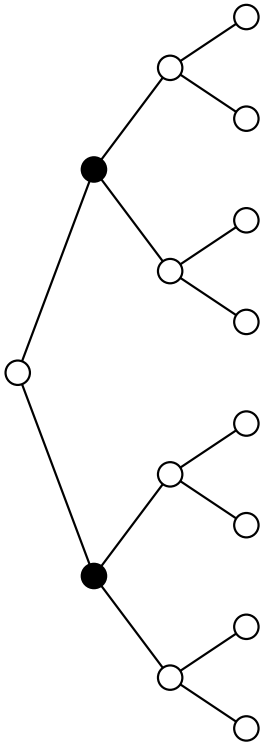


The butterfly property

Frequency Tree



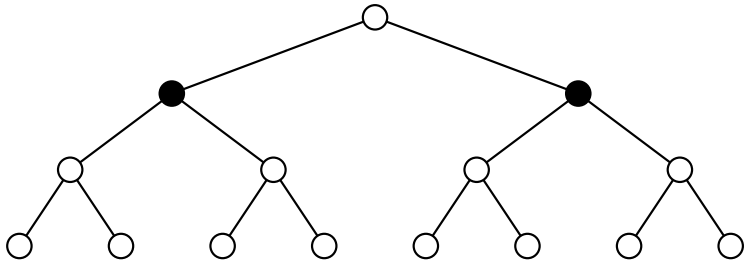
Space Tree



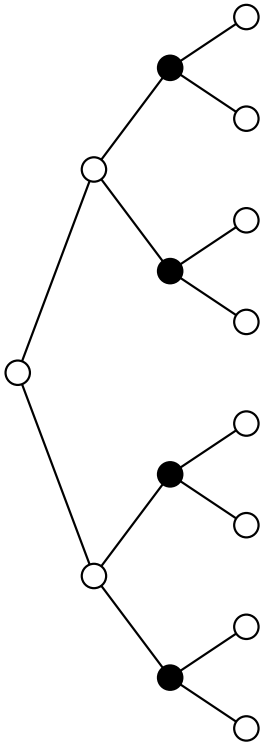
$\Phi_{11}^{(2)}$	$\Phi_{12}^{(2)}$	$\Phi_{13}^{(2)}$	$\Phi_{14}^{(2)}$
$\Phi_{21}^{(2)}$	$\Phi_{22}^{(2)}$	$\Phi_{23}^{(2)}$	$\Phi_{24}^{(2)}$

The butterfly property

Frequency Tree



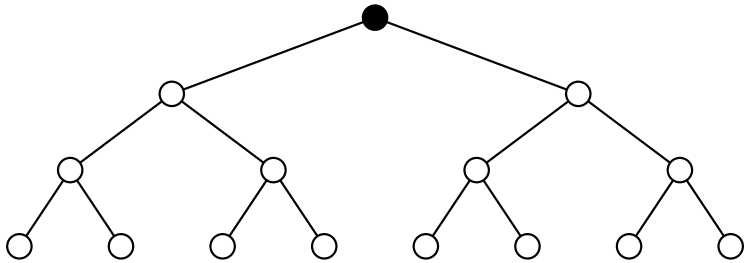
Space Tree



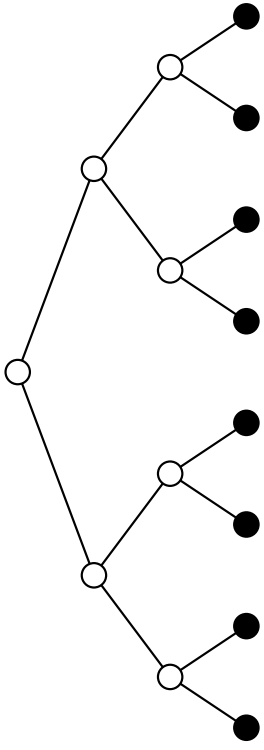
$\Phi_{11}^{(3)}$	$\Phi_{12}^{(3)}$
$\Phi_{21}^{(3)}$	$\Phi_{22}^{(3)}$
$\Phi_{31}^{(3)}$	$\Phi_{32}^{(3)}$
$\Phi_{41}^{(3)}$	$\Phi_{42}^{(3)}$

The butterfly property

Frequency Tree

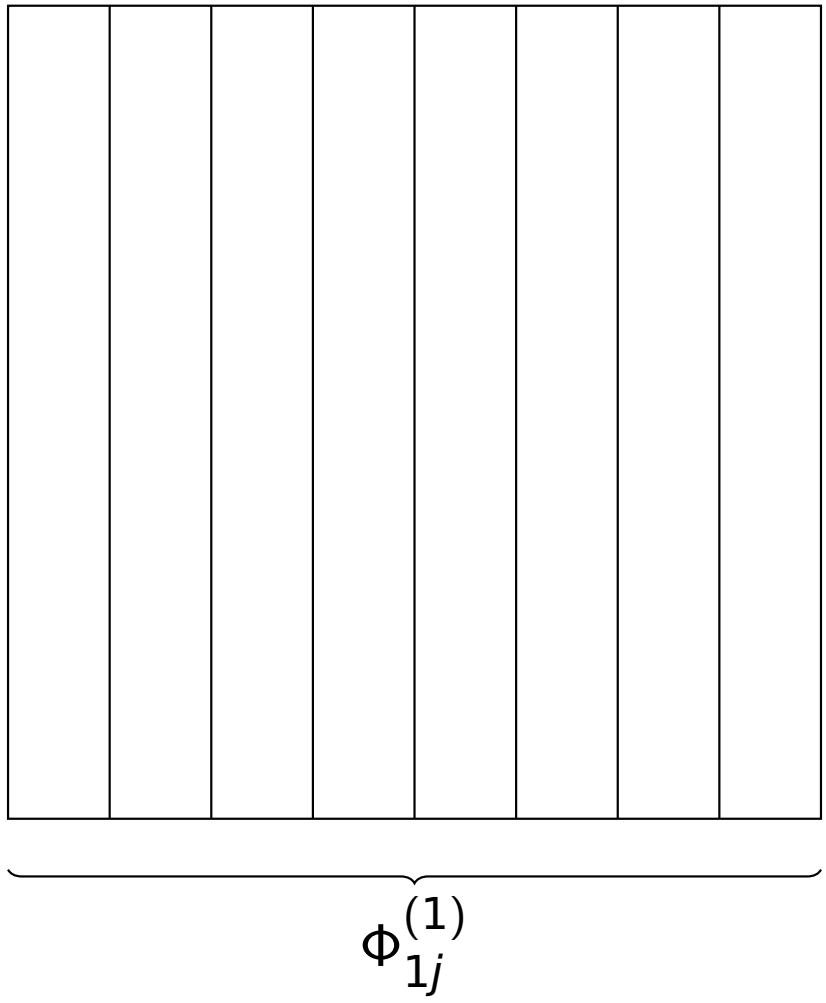


Space Tree

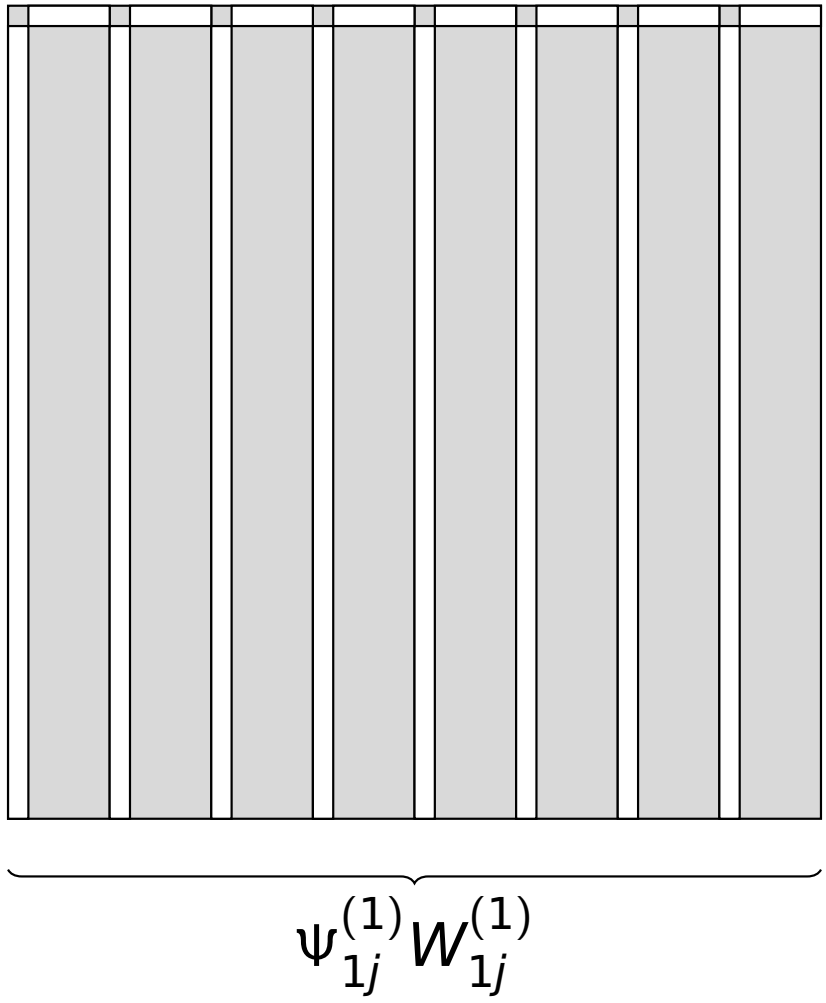


$\Phi_{11}^{(3)}$
$\Phi_{21}^{(3)}$
$\Phi_{31}^{(3)}$
$\Phi_{41}^{(3)}$
$\Phi_{51}^{(3)}$
$\Phi_{61}^{(3)}$
$\Phi_{71}^{(3)}$
$\Phi_{81}^{(3)}$

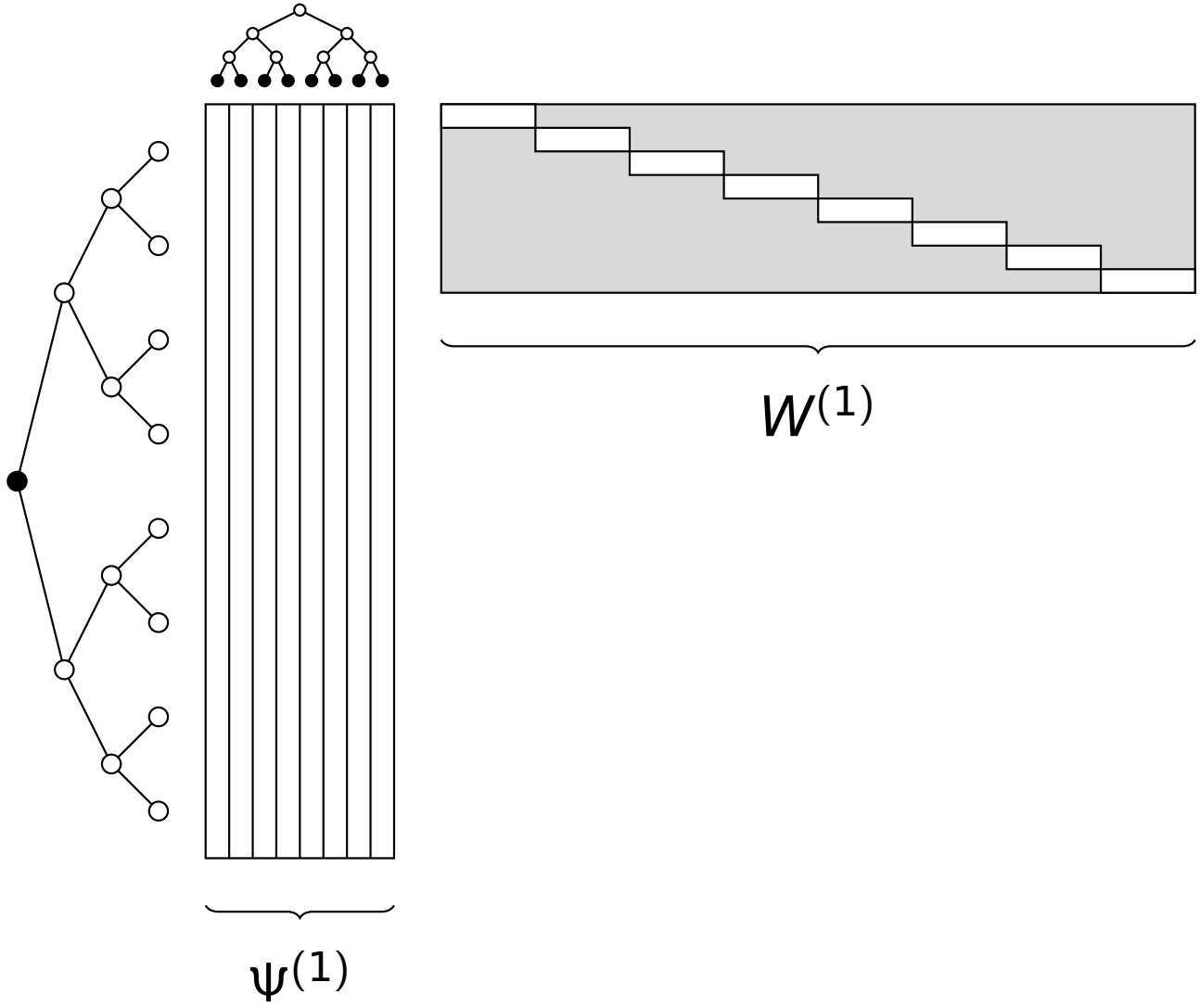
How to compute a column-wise BF



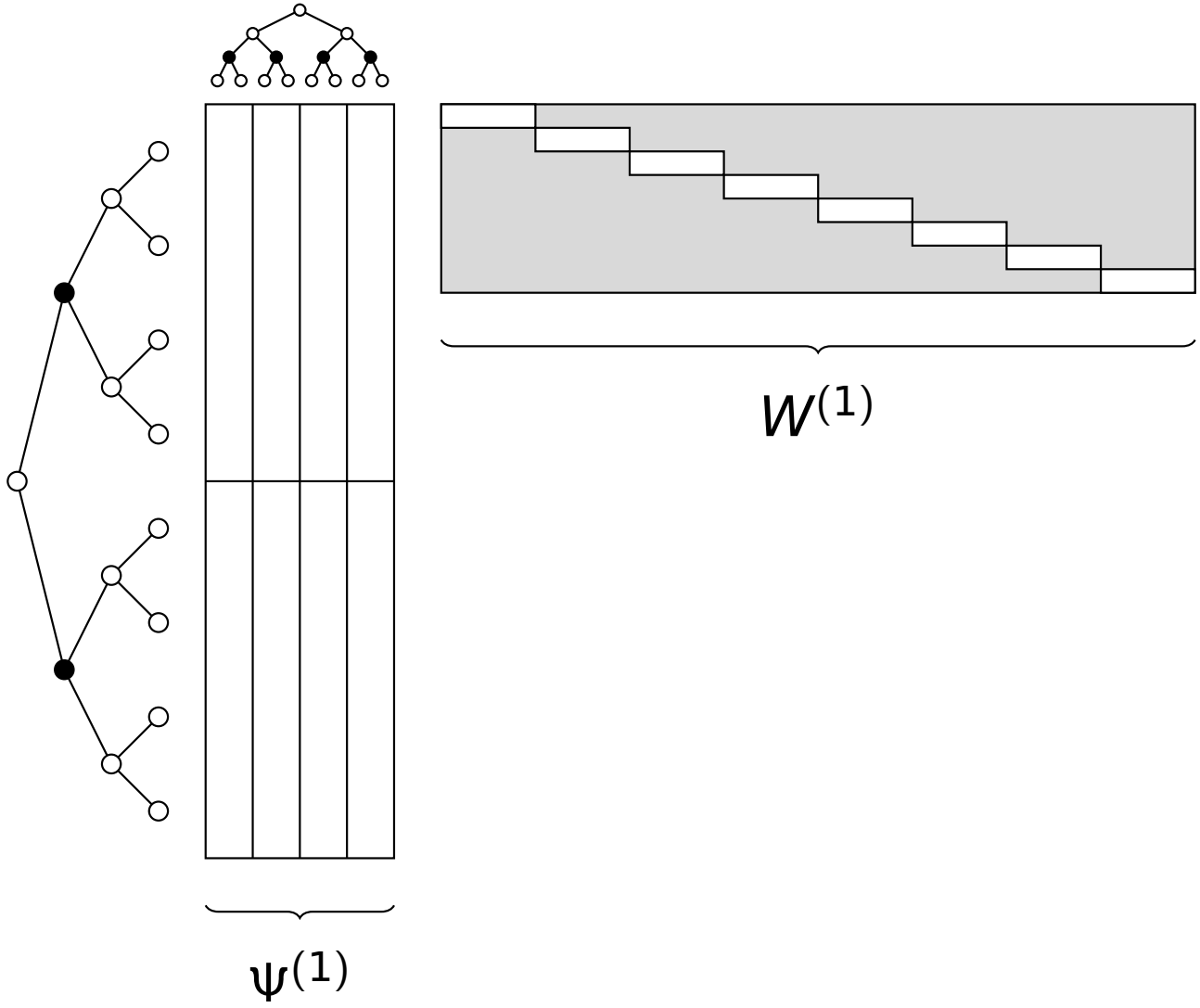
How to compute a column-wise BF



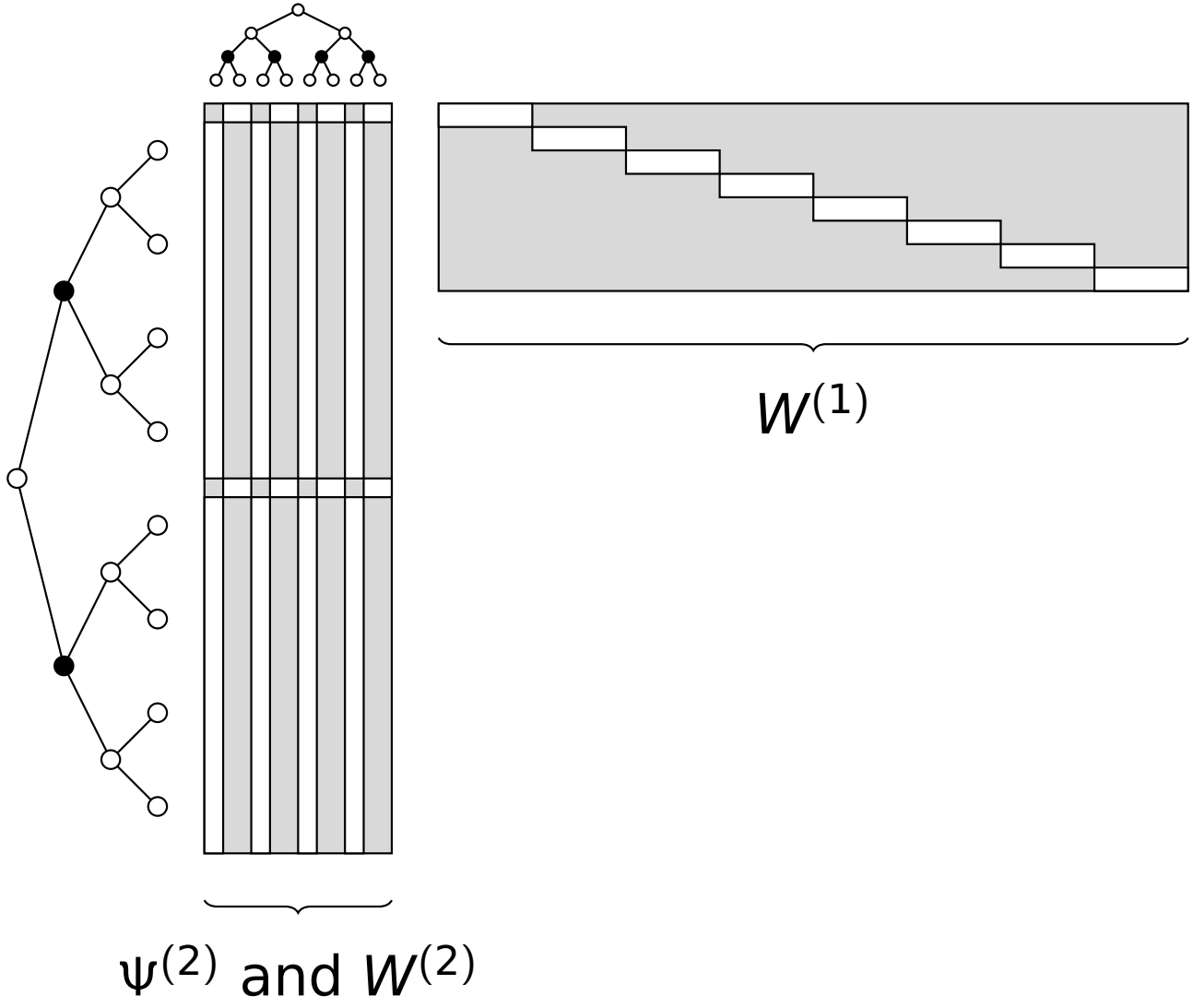
How to compute a column-wise BF



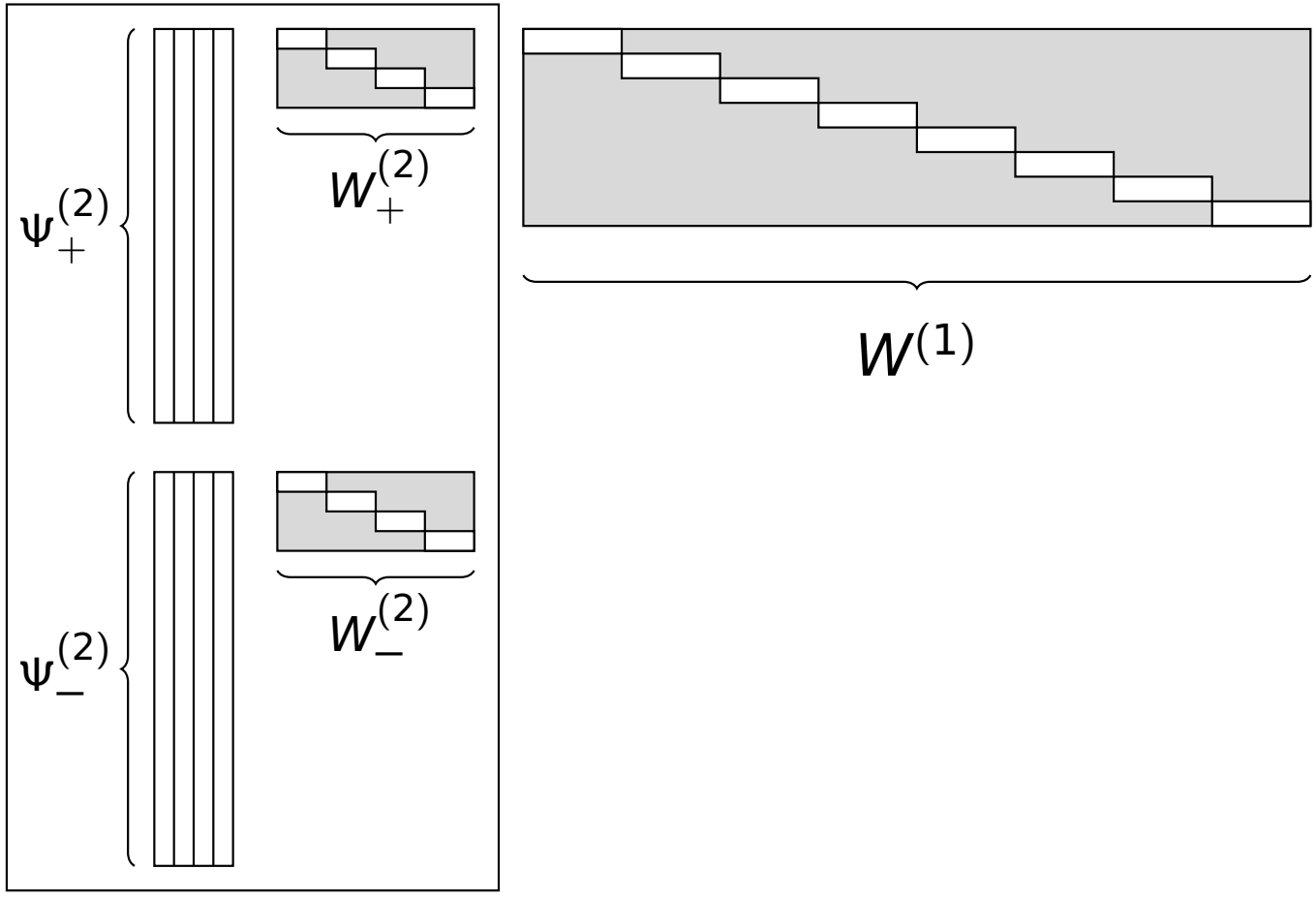
How to compute a column-wise BF



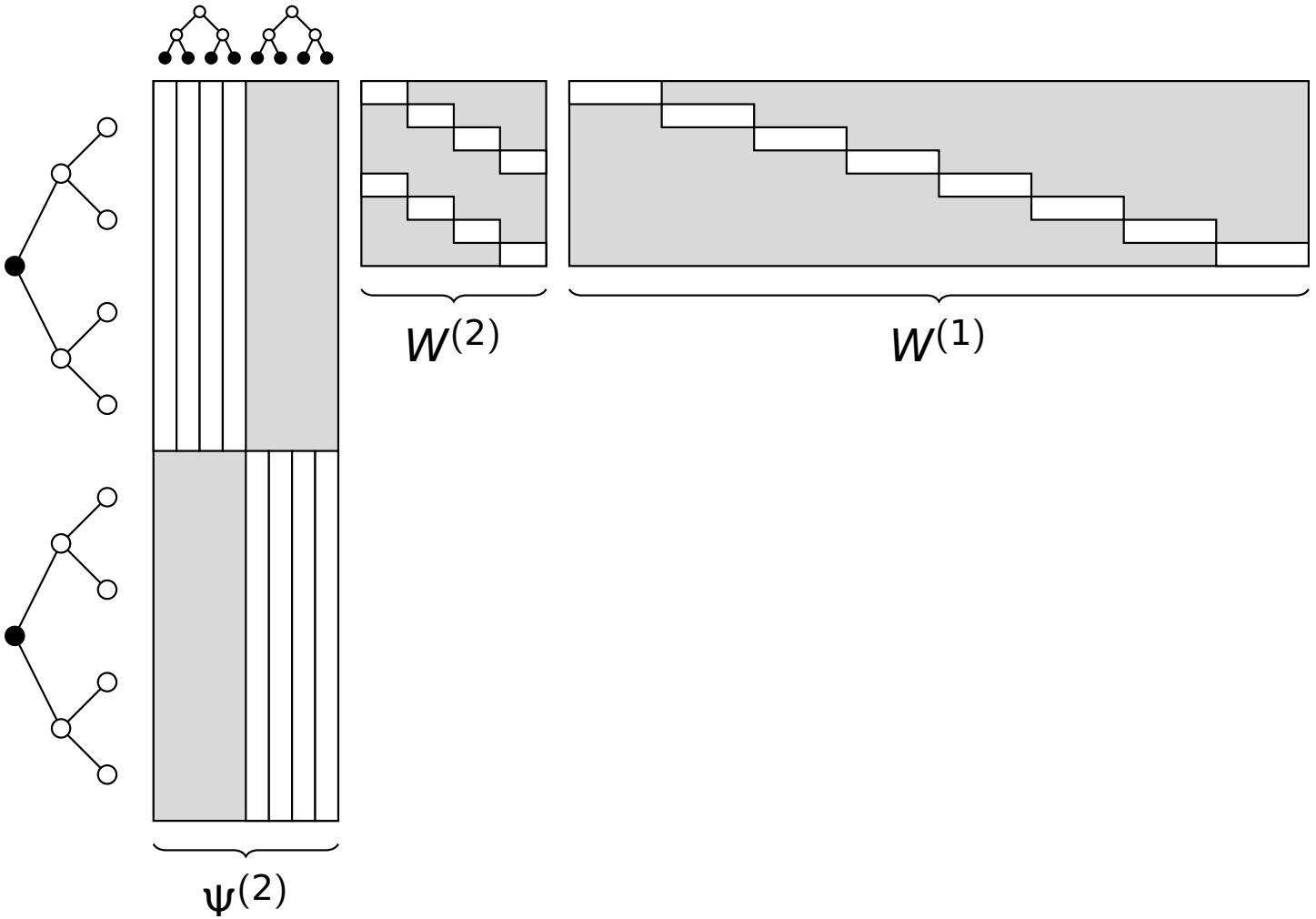
How to compute a column-wise BF



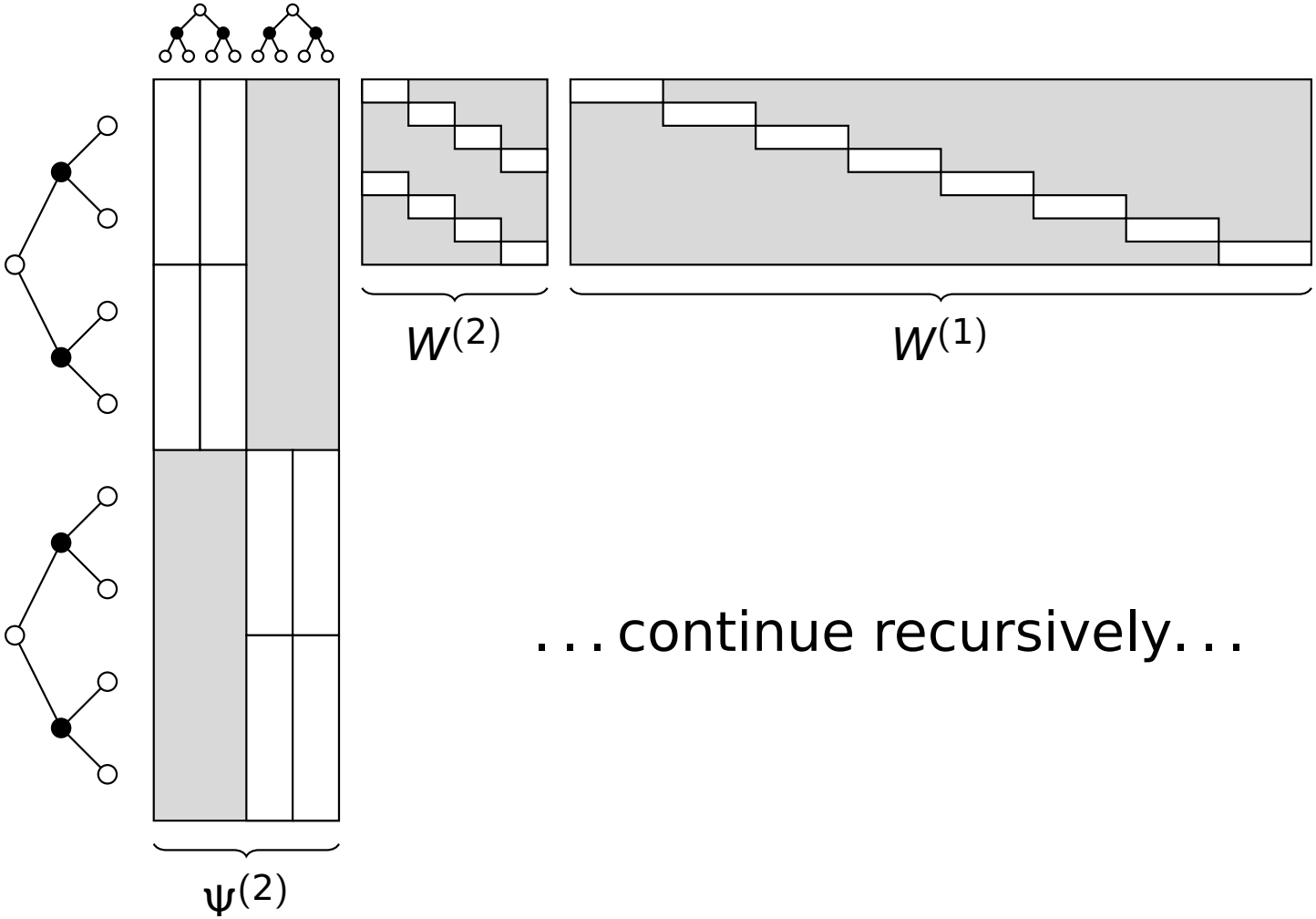
How to compute a column-wise BF



How to compute a column-wise BF

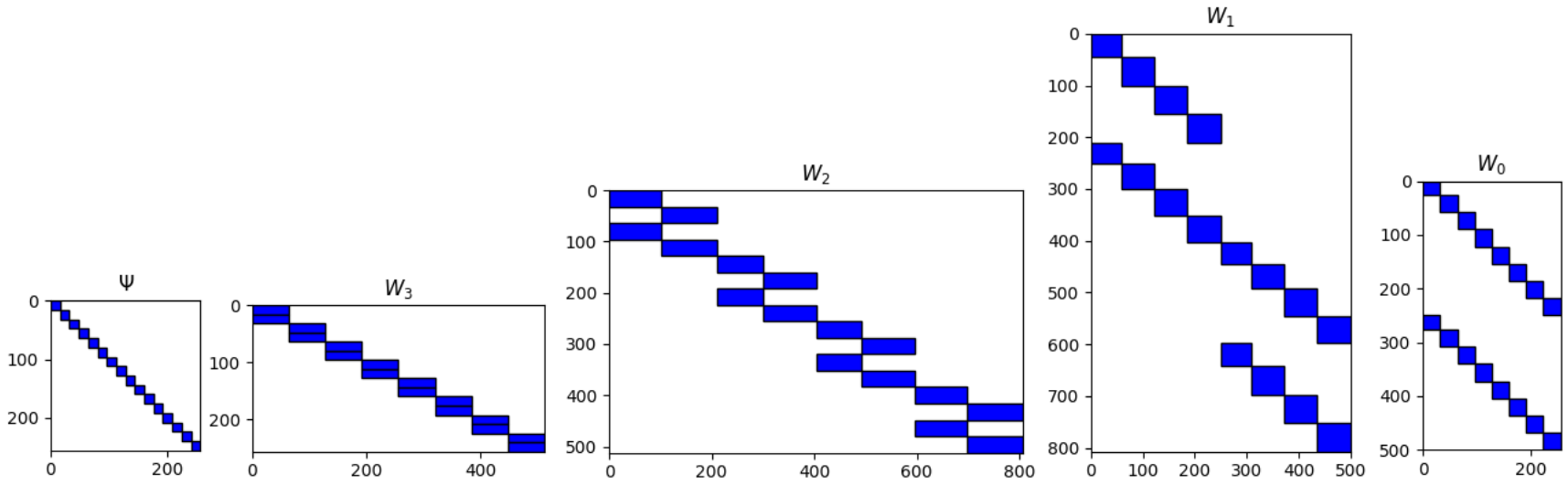


How to compute a column-wise BF



. . . continue recursively. . .

Example of a butterfly factorization



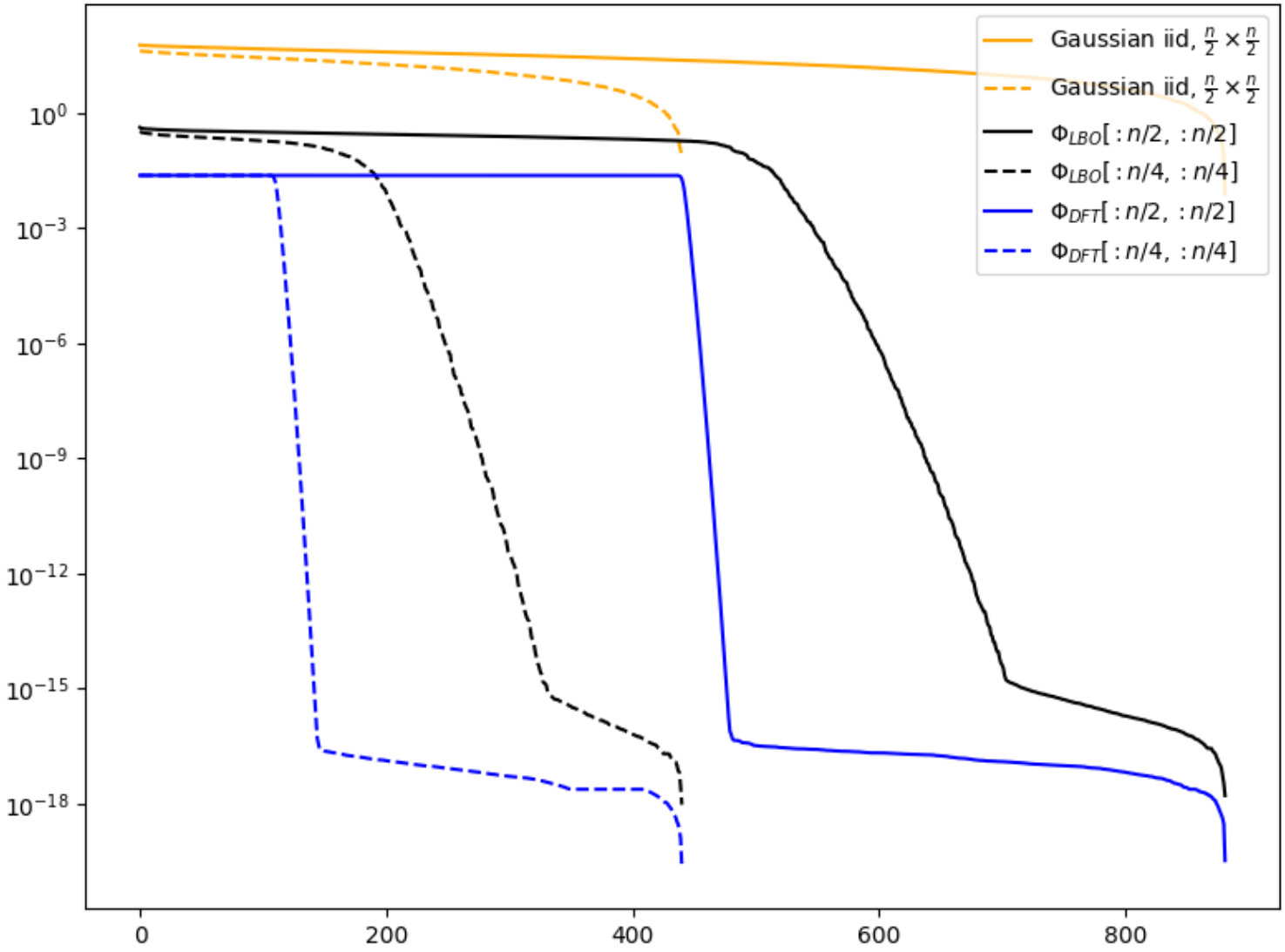
Butterfly factorization of 128×128 discrete Legendre transform:

$$\Phi \approx \Psi^{(3)} W^{(3)} W^{(2)} W^{(1)} W^{(0)}, \qquad \Phi_{ij} = P_j(x_i)$$

Bad choice here because of inner factor size blow-up—butterfly compressed matrix is ~2 larger than original. . .

Can we butterfly surface harmonics?

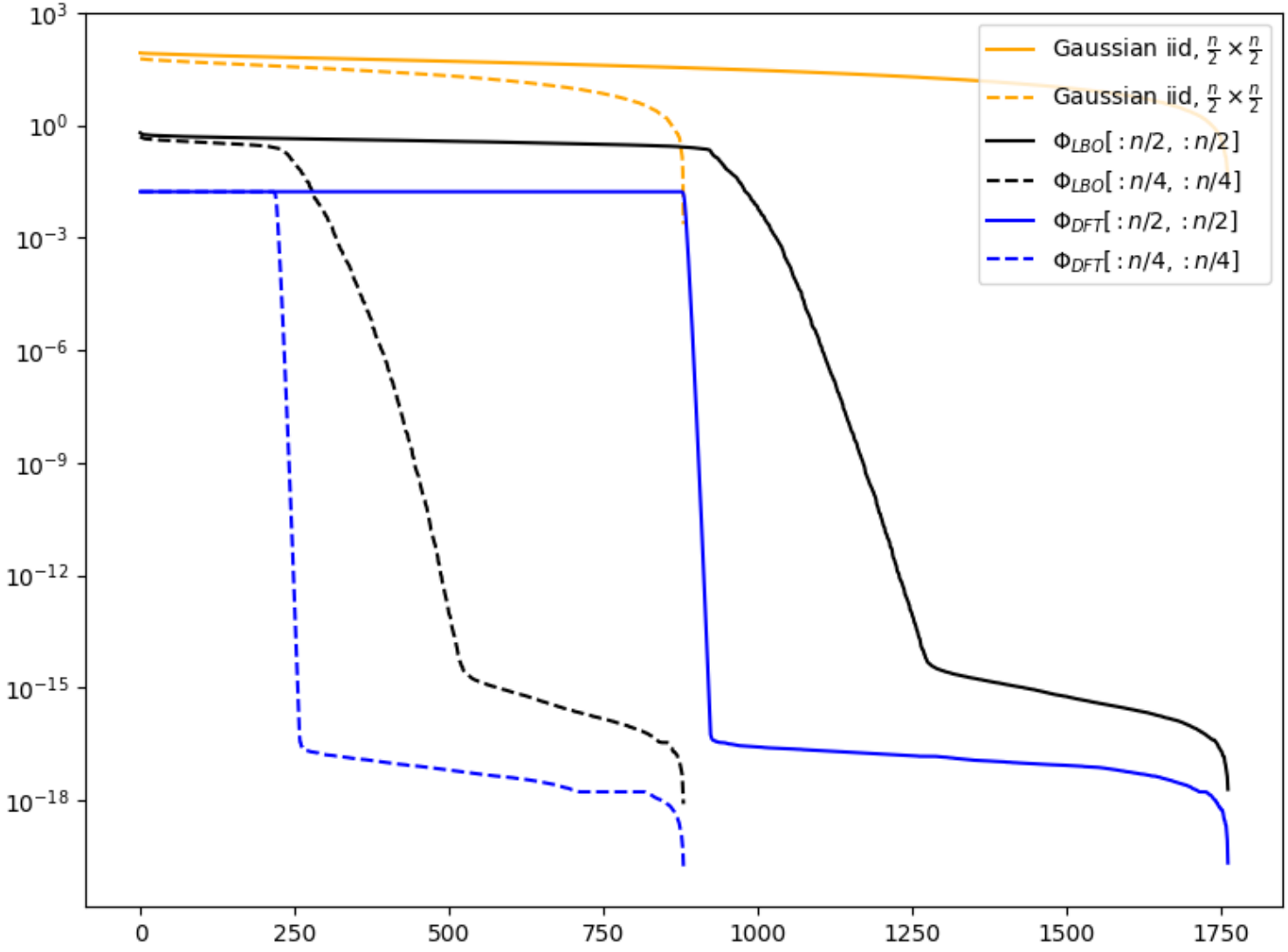
Surface harmonics are *weakly* butterfliable



(“Asian dragon” mesh w/ **1,764** vertices)

Can we butterfly surface harmonics?

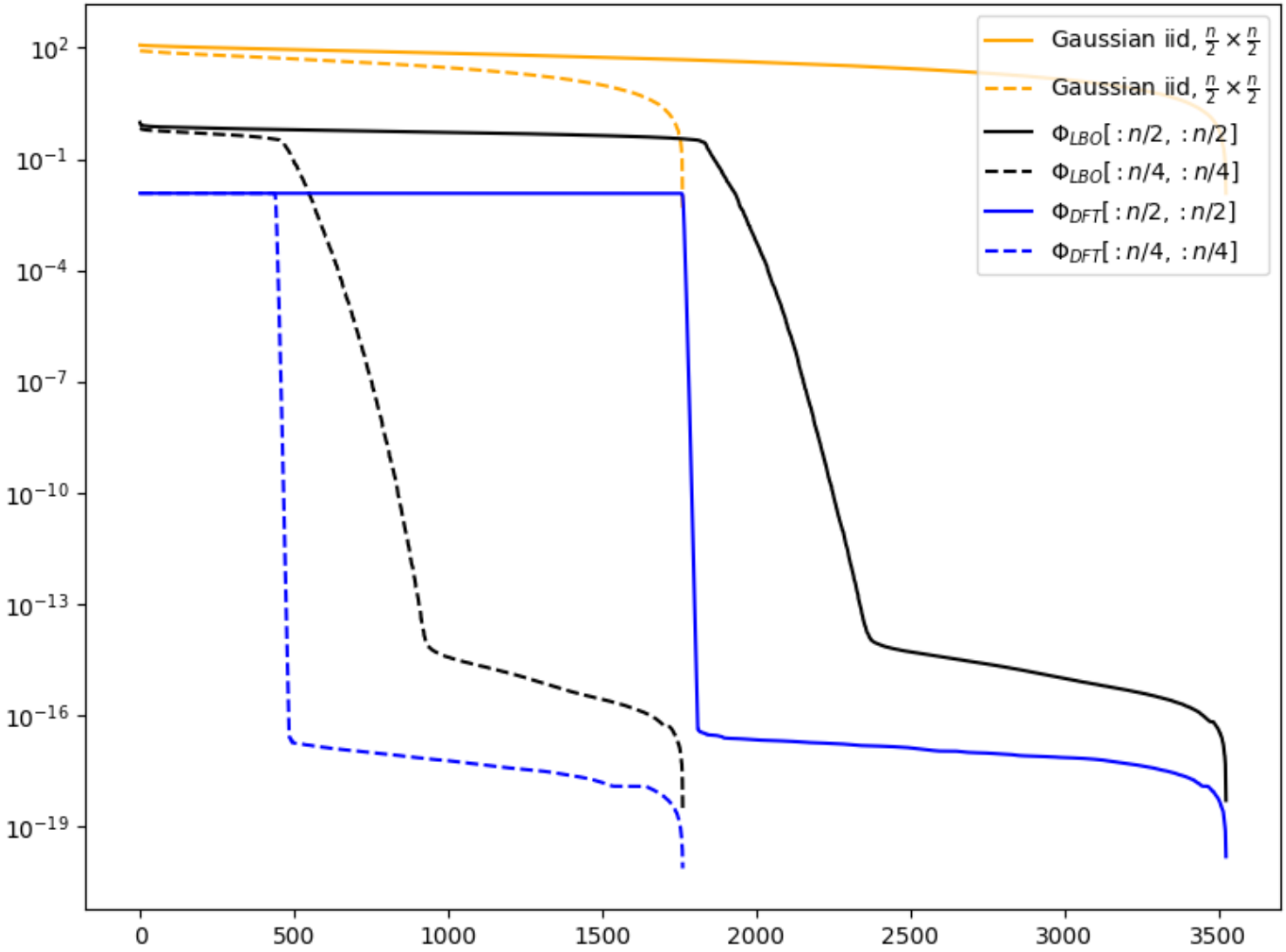
Surface harmonics are *weakly* butterfliable



(“Asian dragon” mesh w/ **3,526** vertices)

Can we butterfly surface harmonics?

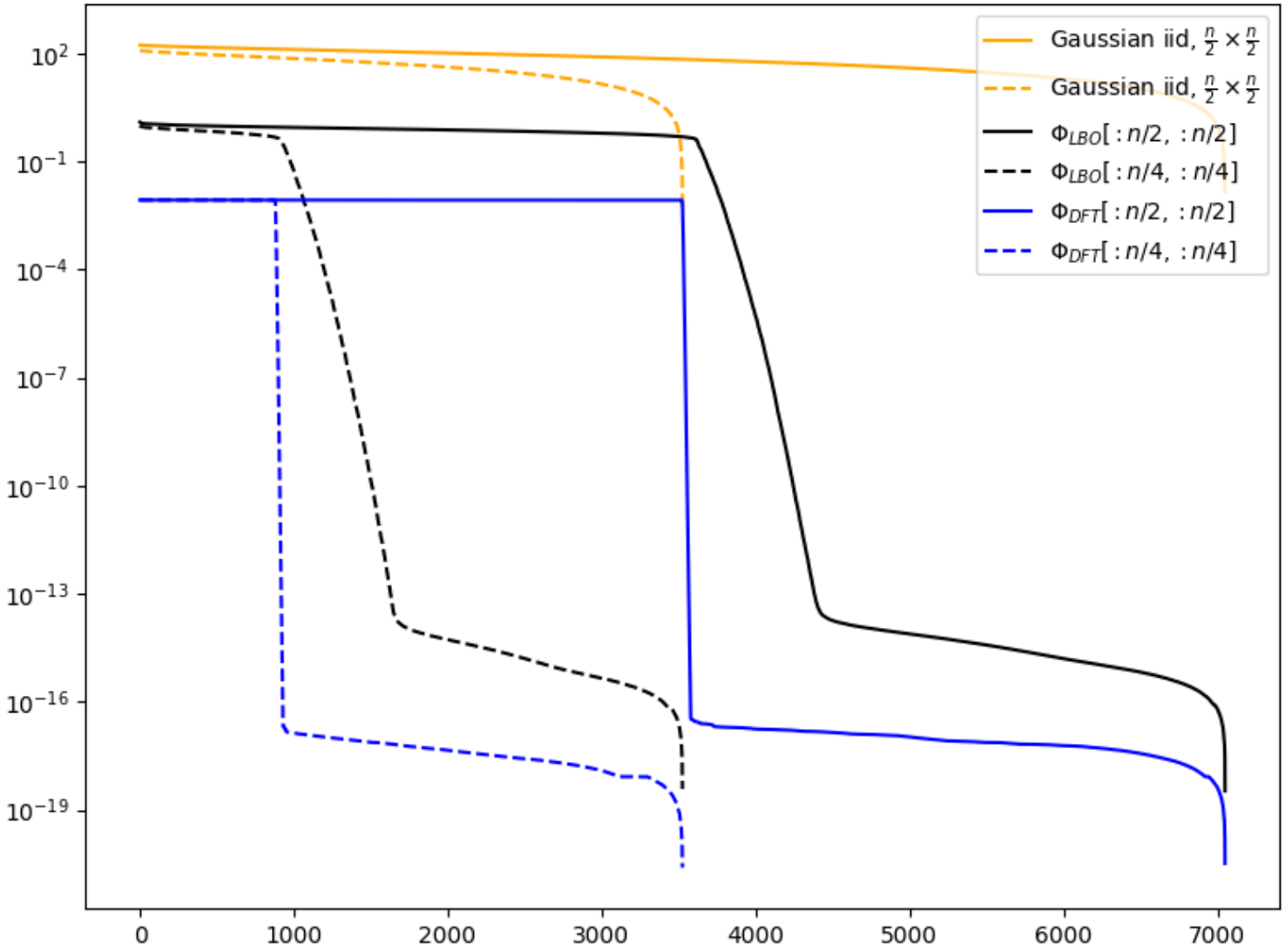
Surface harmonics are *weakly* butterfliable



(“Asian dragon” mesh w/ **7,051** vertices)

Can we butterfly surface harmonics?

Surface harmonics are *weakly* butterfliable



(“Asian dragon” mesh w/ **14,101** vertices)

Can we butterfly surface harmonics?

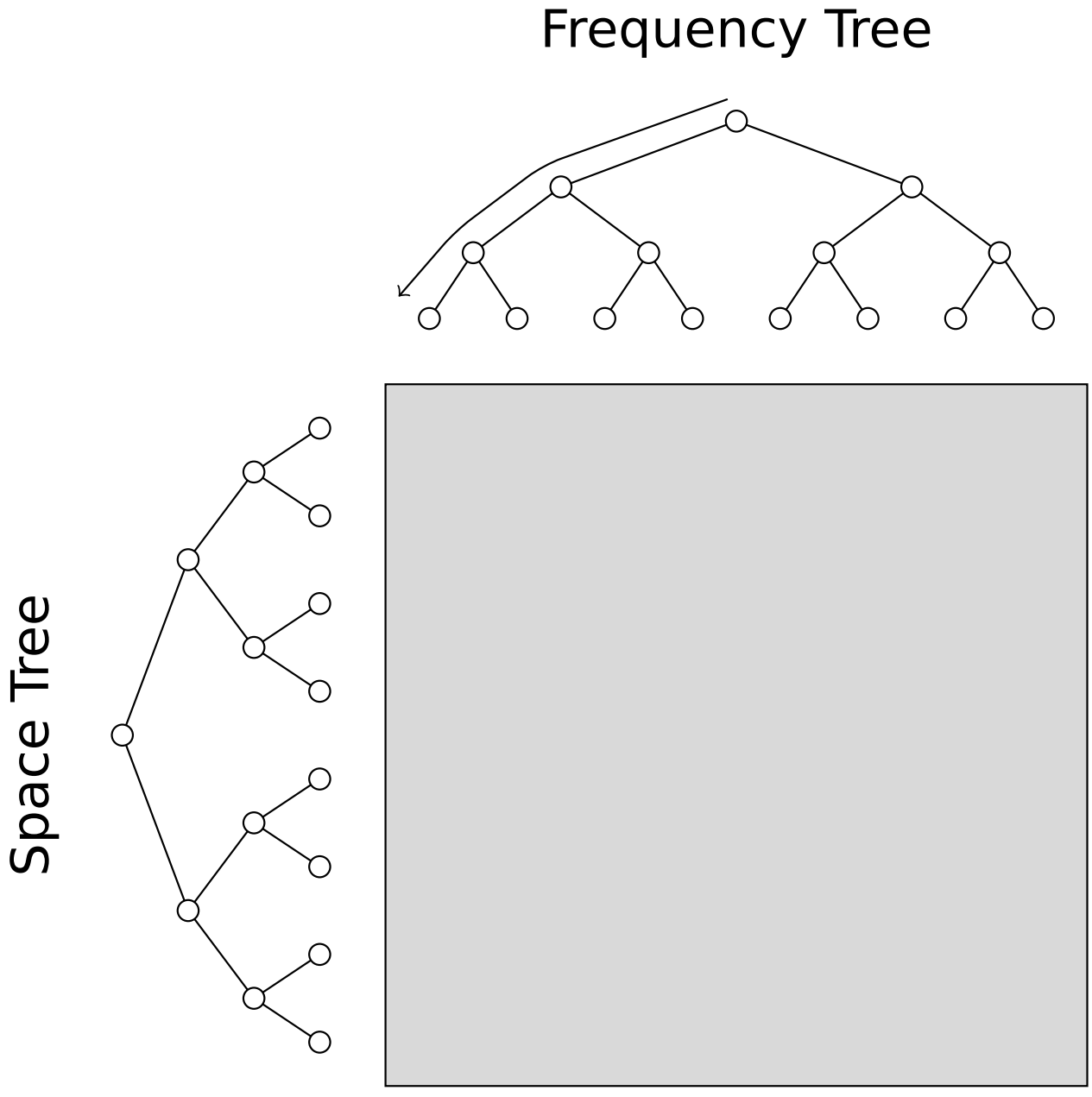
- **Note**: $\omega^2 = \lambda$
- Build tree on uniform grid in frequency—this is a **nonuniform** tree on the eigenvalues
- **Recall**: eigenfunctions of Laplacian on unit sphere are the degree p and order m spherical harmonics Y_p^m and corresponding eigenvalues are $-p(p+1)$
- There are $2p+1$ spherical harmonics of order p...
- More and more eigenfunctions per freq. cell...
- Behavior for LBO on genus-0 surface is similar

Yes—but we can't expect $O(n \log n)$ complexity...

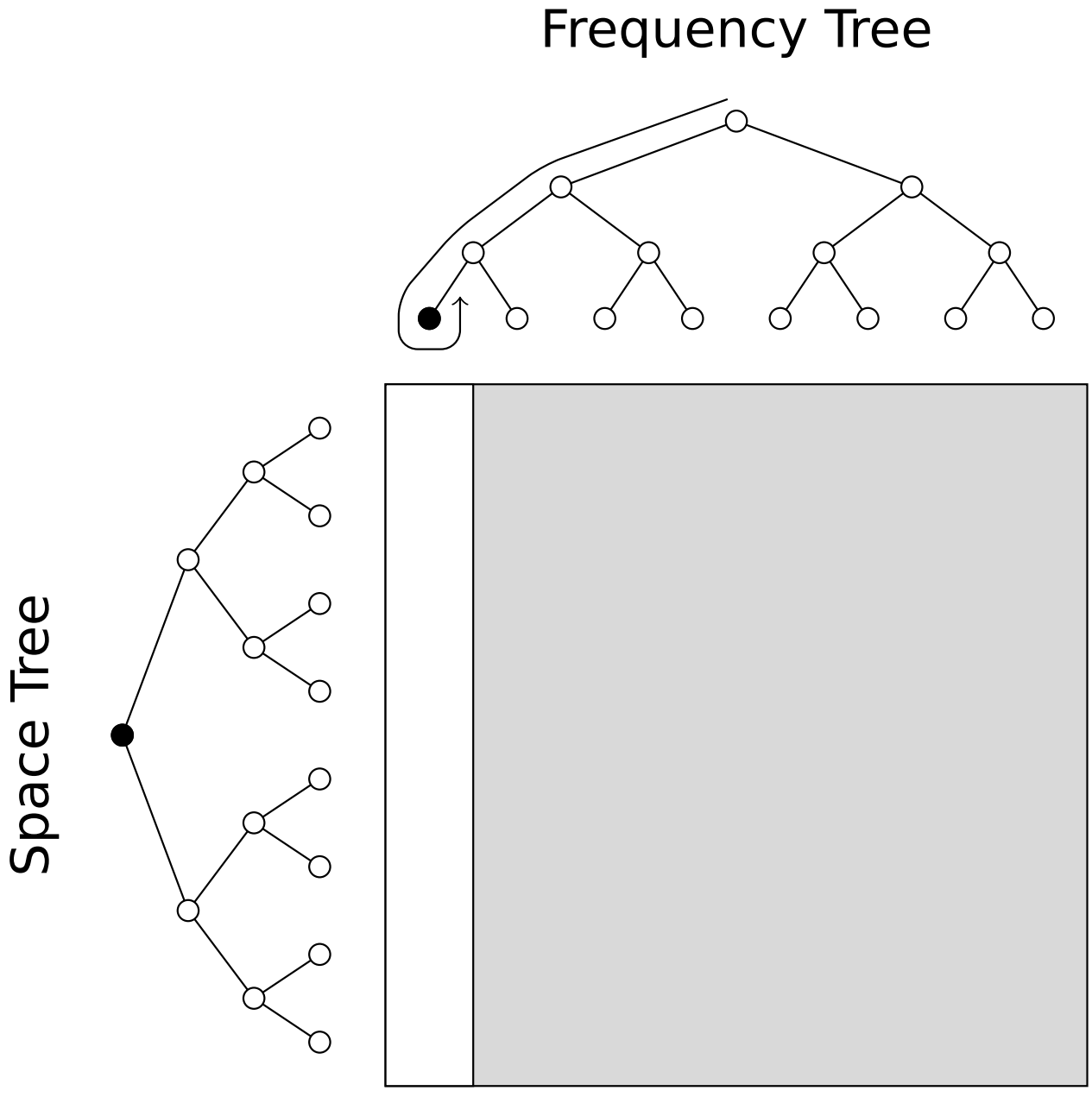
Streaming a butterfly factorization

- Our algorithm for computing the butterfly factorization has the drawback that it requires $O(N^2)$ space to store the eigenvector matrix as it is compressed
- Instead, it's possible to *stream* the butterfly factorization on the fly, using little additional space beyond what is required to store the factorization itself

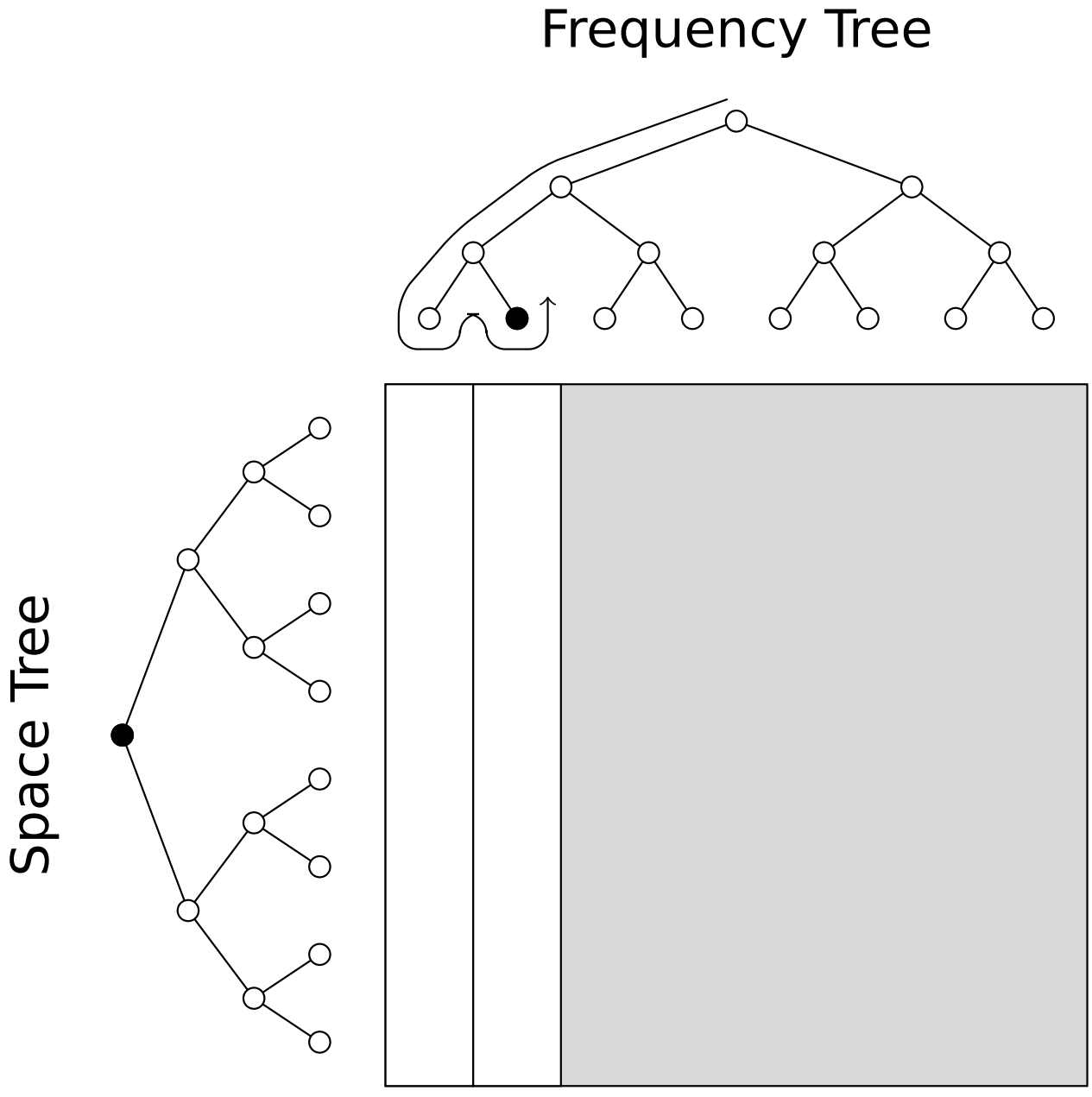
Streaming a butterfly factorization



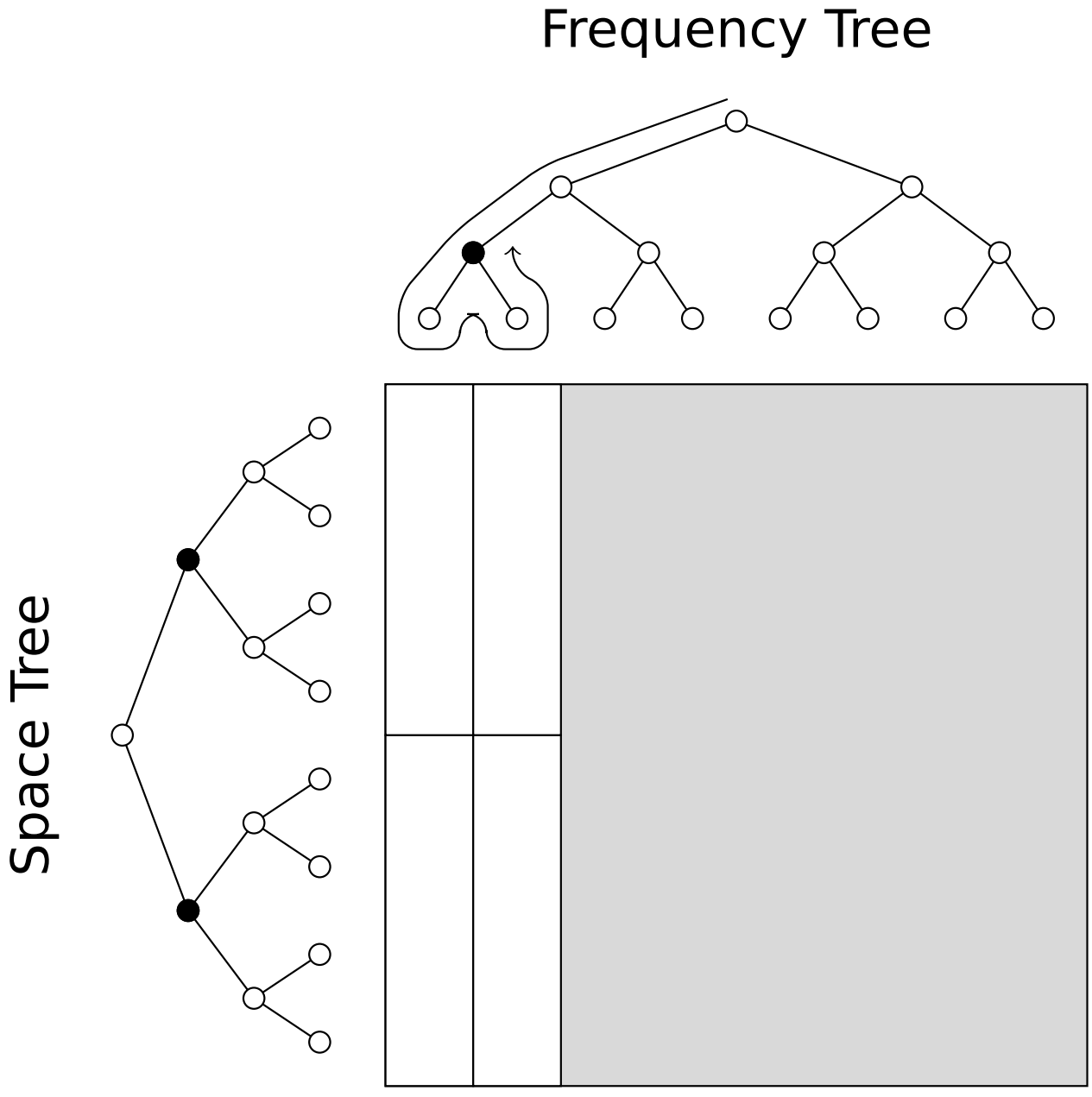
Streaming a butterfly factorization



Streaming a butterfly factorization



Streaming a butterfly factorization



Streaming eigenbands

- Cheap to compute $\lambda_{\min}$ and $\lambda_{\max}$.
- Solve shifted generalized eigenvalue problem:

$$(L - \mu M)u = (\lambda - \mu)Mu$$

  to recover eigenvalues nearest μ first.
- Set up grid of eigenvalues:

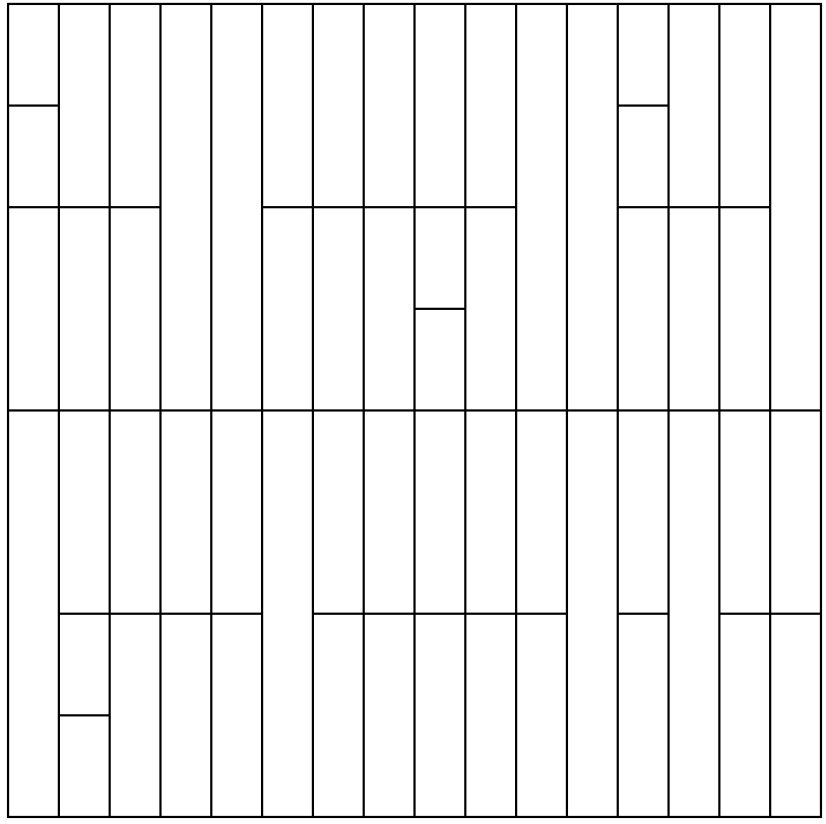
$$\tilde{\lambda}_0 = \lambda_{\min} < \tilde{\lambda}_1 < \cdots < \tilde{\lambda}_{N-1} < \tilde{\lambda}_N = \lambda_{\max}$$

- **Algorithm:** for each interval $[\tilde{\lambda}_{j-1}, \tilde{\lambda}_j)$, guess number of eigs. in band: $k > 0$. Compute first k eigenpairs. Finished if eigenvalues cover the band. Otherwise, set $k \to 2k$ and try again.

Compute p eigenpairs in roughly $O(np)$ time. Can be used to stream bands one at a time.

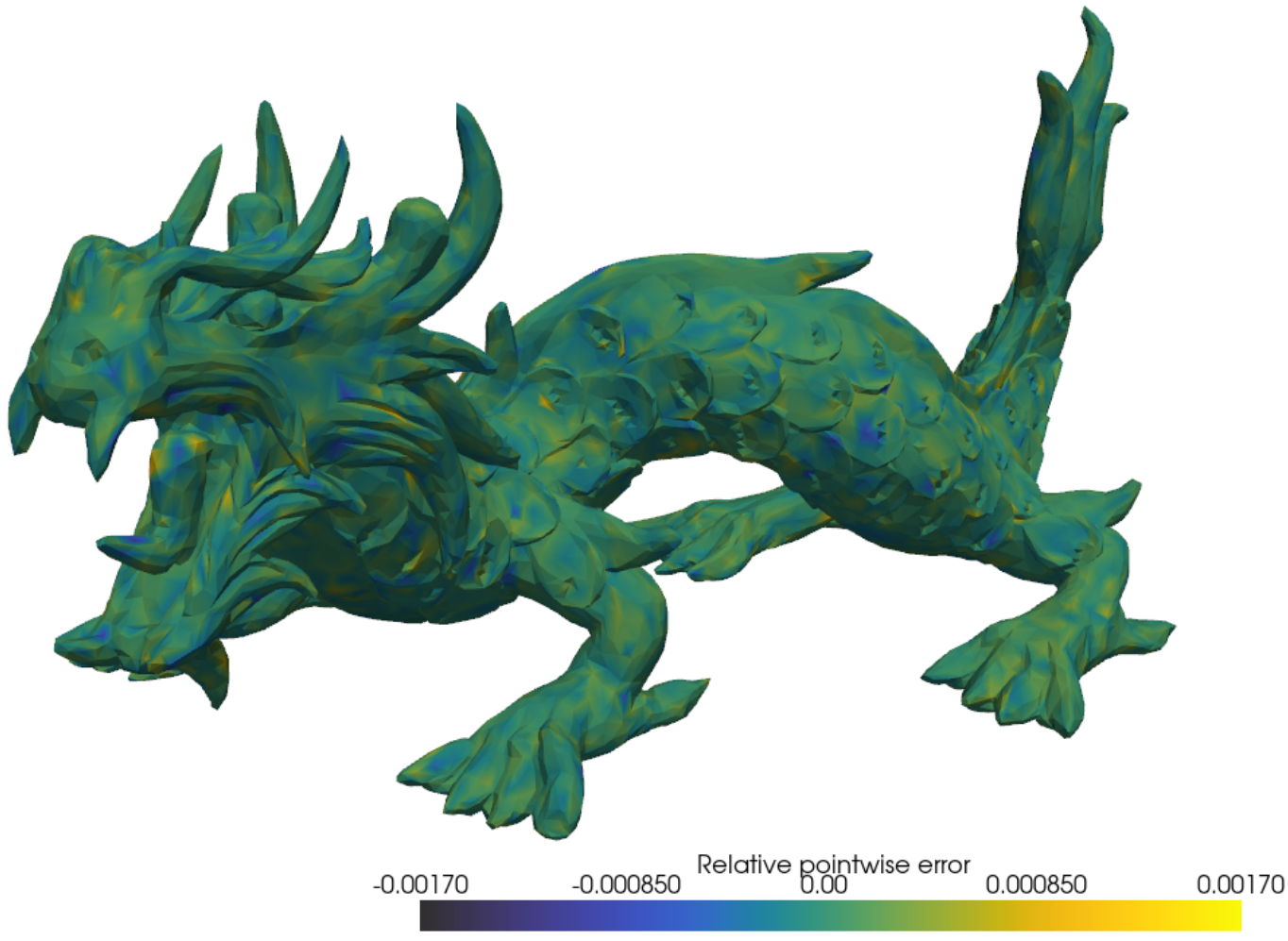
Adaptive butterfly factorization

For each column, find the highest cut through the spatial tree such that each subblock can be truncated with the desired precision.



Numerical results

For low-rank factorizations, use truncated SVD, keeping singular vectors where $\sigma_k \geq \epsilon \sigma_1$.

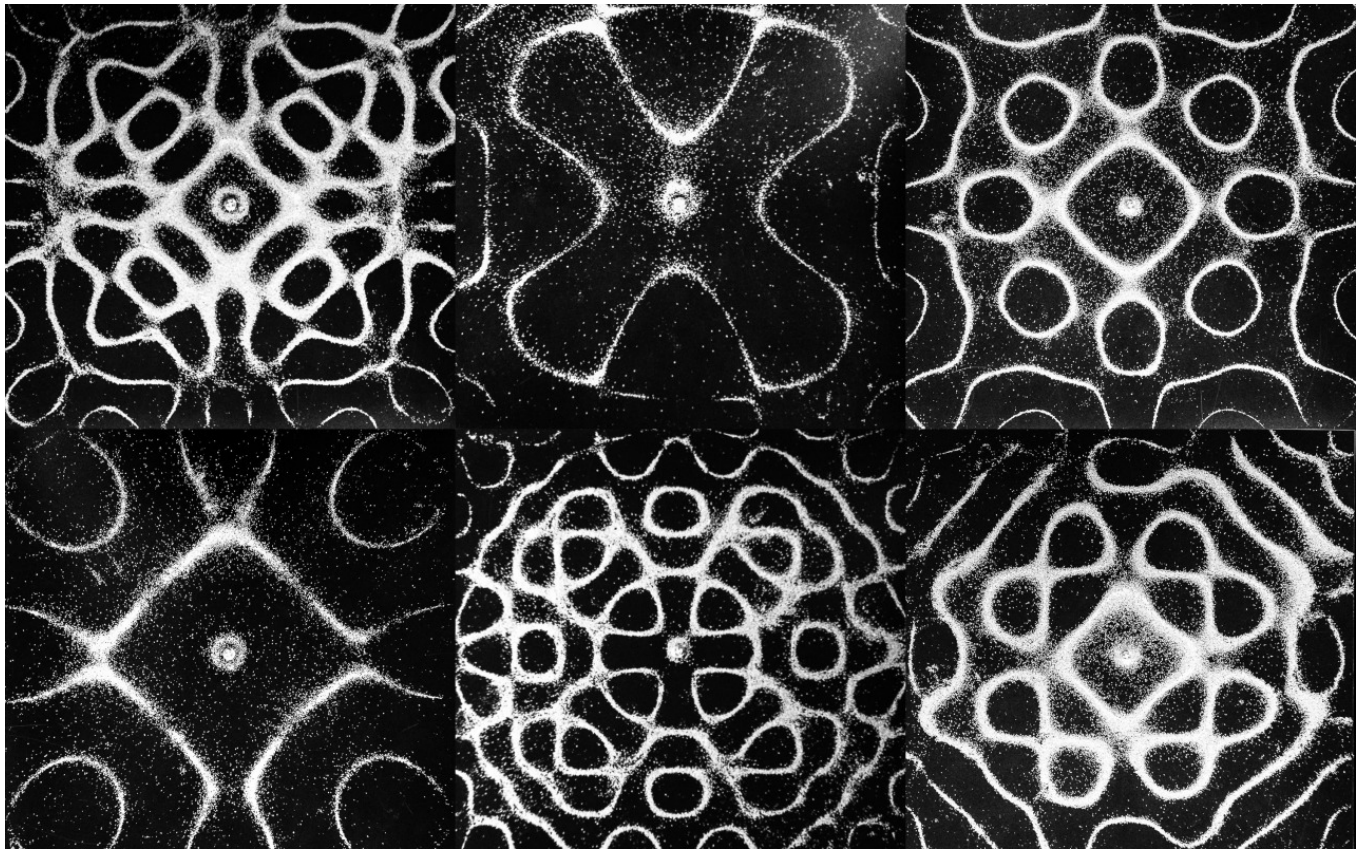


Numerical results

Results for adaptive BF with relative error $\epsilon = 10^{-3}$, computing lowest $\frac{1}{16}$th of spectrum.

Verts	Eigs	Orig	BF	Ratio	Order
1,764	110	1.5 MB	0.9 MB	1.67	
3,526	220	5.9 MB	3.0 MB	1.97	1.74
7,051	440	23.7 MB	9.7 MB	2.44	1.69
14.1K	881	94.8 MB	30.2 MB	3.14	1.64
28.2K	1,762	379.1 MB	92.8 MB	4.09	1.62
56.4K	3,524	1.5 GB	278.7 MB	5.44	**1.59**
113K	7,049	6.0 GB	863.7 MB	7.02	1.63
225K	14.1K	24.2 GB	2.5 GB	9.42	**1.58**

Current work: the Fiedler tree



Courant's Nodal Domain Theorem: zero level set of *k*th eigenfunction of Laplace-Beltrami operator partitions Ω into at most *k* pieces ("nodal domains").

Source: http://dataphys.org/list/chladni-plates/

Current work: the Fiedler tree

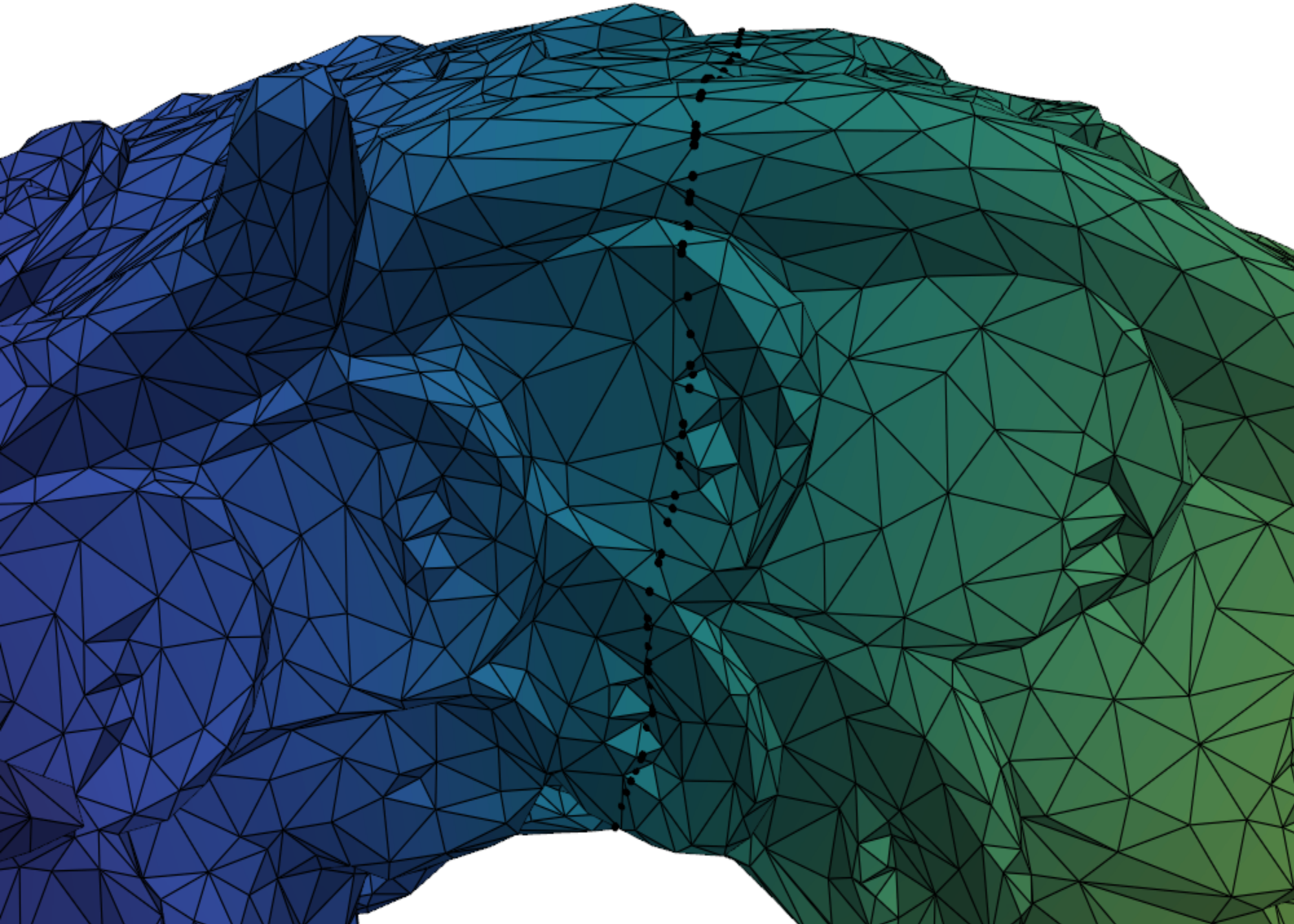
Fiedler vector is the eigenvector for first nonzero eig.
Zero level set separates into two nodal domains.



Current work: the Fiedler tree

So far have observed better compression than with octree, but need to insert new vertices into the mesh.





Current work: the Fiedler tree

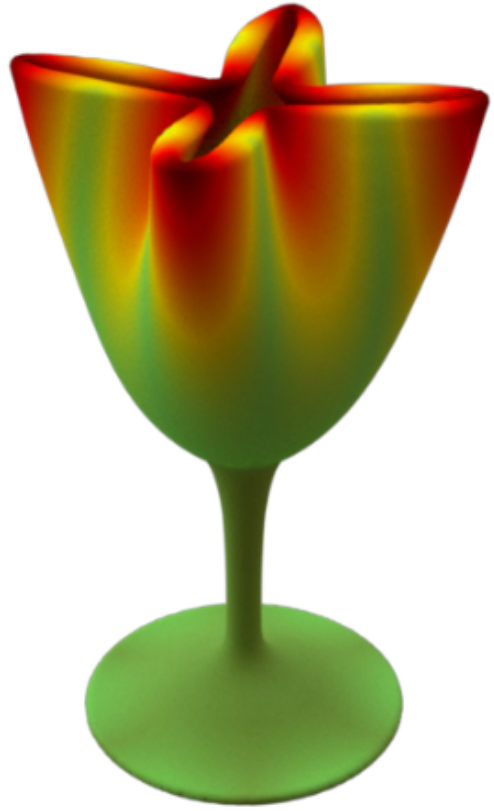
Questions:

- How to fully exploit the Courant nodal domain theorem?
- How to design optimal space trees for higher genus and higher dimension manifolds?
- How to efficiently construct Fiedler trees?

Application: acoustic mode compression



Mode 3, 1.9 kHz



Mode 15, 9.3 kHz



Mode 33, 16 kHz

Source: Langlois et al., “Eigenmode Compression for Modal Sound Models”

Application: eigenfluids

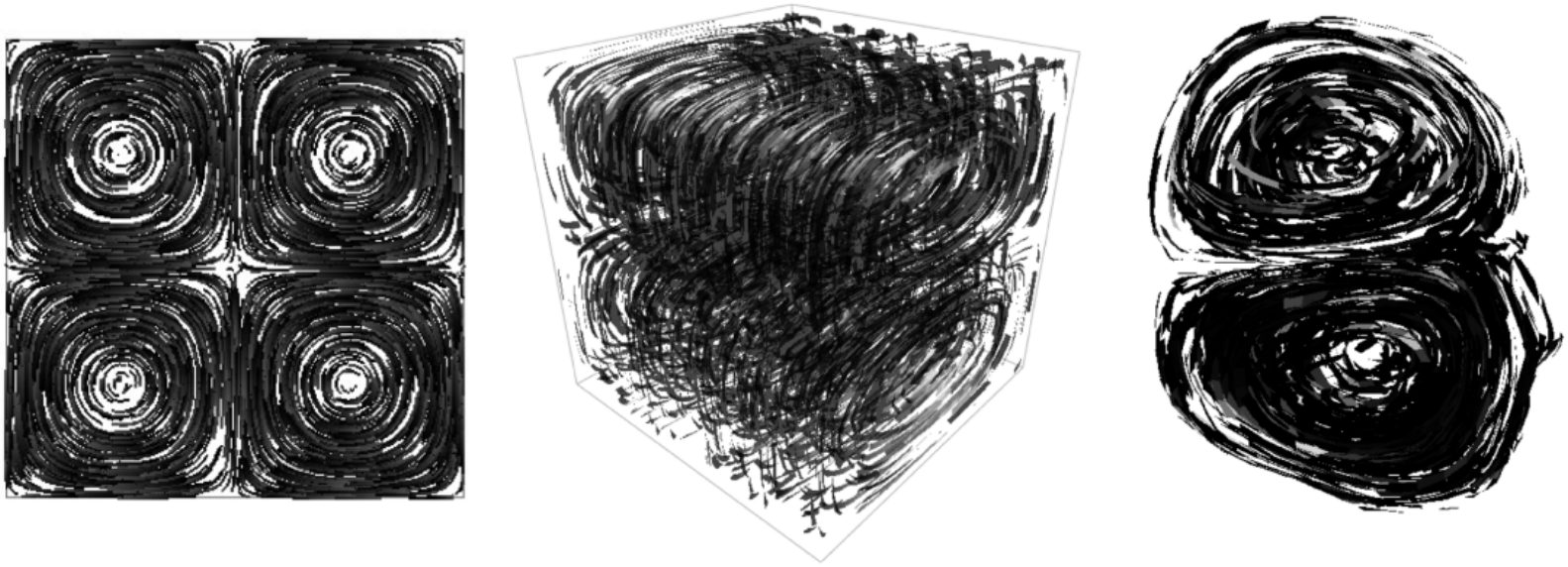


Fig. 1. Examples of Laplacian eigenfunction velocity basis fields for various domains. They are stationary solutions to the Navier-Stokes equations. For simple domains Laplacian eigenfunctions have closed-form expressions while for general meshes they are defined through eigendecomposition of a discrete Laplace operator.

Source: De Witt et al., “Fluid Simulation Using Laplacian Eigenfunctions”

Conclusion

- The Laplace-Beltrami operator on a surface is “weakly” butterfly-factorizable, giving an **asymptotically faster transform**.
- Adaptive butterfly factorization combined with post-order “streaming” algorithm for construction lets us build it using **optimal space complexity**.
- **SFP**, Tim Langlois, Mike O’Neil. “Butterfly-accelerated manifold harmonic transforms.” *In preparation.*

Thank you!